

Universidad de Buenos Aires
Facultad de Ciencias Exactas y Naturales

Monitoreo con drones asistidos por un centro de recarga móvil: una variante discreta

Tesis de Licenciatura en Ciencias de Datos

Leonard Ehrhorn

Director: Diego Delle Donne
Codirector: Guillermo Duran
Buenos Aires, 2025

MONITOREO CON DRONES ASISTIDOS POR UN CENTRO DE RECARGA MÓVIL: UNA VARIANTE DISCRETA

En esta tesis exploramos la investigación existente sobre problemas de optimización de operaciones con UAVs (Unmanned Aerial Vehicles). Luego describimos un trabajo de la literatura sobre operaciones de monitoreo coordinadas entre drones y un vehículo base, donde se plantea una solución mediante programación convexa. Construimos una versión discretizada de este problema y lo resolvimos mediante un modelo de Programación Lineal Entera Mixta (MILP), con diferentes alternativas según cómo se mueve el vehículo base. Luego comprobamos experimentalmente el desempeño de nuestro modelo, comparando los resultados para las diferentes alternativas y discretizaciones con diferentes grados de refinamiento. Encontramos que uno de los modelos alternativos planteados parece más eficaz que los otros, aunque el problema planteado es NP-Hard y en muchas de las instancias estudiadas no se encontró solución óptima dentro del tiempo límite. Hemos examinado los tiempos de construcción computacional de los modelos, y sus tiempos de resolución en los casos donde termina a tiempo. Con estos resultados estudiamos la influencia de los diferentes parámetros en los tiempos de construcción y resolución, y planteamos posibles alternativas para intentar mejorarlos.

Palabras claves: Unmanned Aerial Vehicle, base, MILP, grafo, grilla, discretización.

AGRADECIMIENTOS

A mi director, Diego Delle Donne, por su interés y compromiso con la tesis y todos los secretos del área que me enseñó en el trayecto.

Al codirector, Guillermo Duran, por haber recomendado al director y el tiempo invertido para acompañar el proceso.

A Andrés Farall por su acompañamiento sostenido como coordinador de la materia Tesis, y a Nazareno Faillace por haber aceptado ser jurado.

A Dante, mi amigo de la primaria, única persona ajena a la Facultad con la que hablamos el mismo idioma.

A mi familia, por haberme bancado todo el tiempo necesario.

A mi gata, mi mejor amiga mientras vivió, por haber crecido conmigo.

A la militancia, una parte indispensable de mi vida.

A la Universidad Pública y el sistema científico, por continuar dándolo todo y ser de primer nivel.

Índice general

1.. Introducción 1

2.. Estado del arte de la optimización con drones 3
2.1. Cobertura y monitoreo con UAVs 3
2.1.1. Problemas de cobertura total 3
2.1.2. Problemas de cobertura parcial 4
2.1.3. Operaciones de búsqueda 4
2.2. Operaciones combinadas entre drones y vehículos base 4
2.2.1. Problemas clásicos relacionados 5
2.2.2. Modelos con vehículos complementarios y bases de recarga 6

3.. Construcción del modelo discreto 9
3.1. Descripción del problema 9
3.2. Formulación matemática 12
3.2.1. Restricciones elementales 13
3.2.2. Restricciones de conectividad para los drones 15
3.2.3. Conectividad para el vehículo base: grilla damero o completa 16
3.2.4. Reducción de soluciones redundantes 20
3.2.5. Función objetivo 21

4.. Experimentación 23
4.1. Construcción de las instancias 23
4.2. Resultados y análisis 26

5.. Conclusiones 35

1. INTRODUCCIÓN

Los UAVs (Unmanned Aerial Vehicles), conocidos en castellano como drones, pueden ser utilizados para muy diversas aplicaciones civiles y comerciales. La evolución de esta tecnología en las últimas décadas permite tenerlos en cuenta para mejorar la eficiencia de procesos como búsqueda, monitoreo, agricultura y transporte [Otto et al., 2018].

Nuestra principal referencia en este trabajo será un paper de Amorosi, Puerto y Valverde [Amorosi et al., 2023]. En el mismo se presenta un problema de monitoreo de zonas con drones, ayudados por un centro de recarga móvil. Se encara el problema con un modelo de programación convexa y una heurística, y se toma como campo de aplicación una operación de monitoreo de zonas con alta concentración de personas durante un evento desarrollado en tiempos de cuarentena por el COVID-19. Más concretamente, se utilizan los métodos desarrollados para proponer un esquema de monitoreo coordinado entre un helicóptero y dos drones en ocasión de la Fiesta de los Patios de Córdoba. El objetivo del monitoreo es prevenir el hacinamiento de personas durante la actividad, observando diferentes zonas al mismo tiempo, mientras se busca reducir el costo de uso del helicóptero y el tiempo que pasa volando sobre zonas con alta concentración de gente. En la Figura 1.1, tomada del paper en cuestión, se puede observar el mapa de grafos cuya exploración se requiere. Estos grafos representan los 6 caminos donde el festival se desarrolla con actividades simultáneas. Nosotros atacaremos el problema de coordinación de drones y

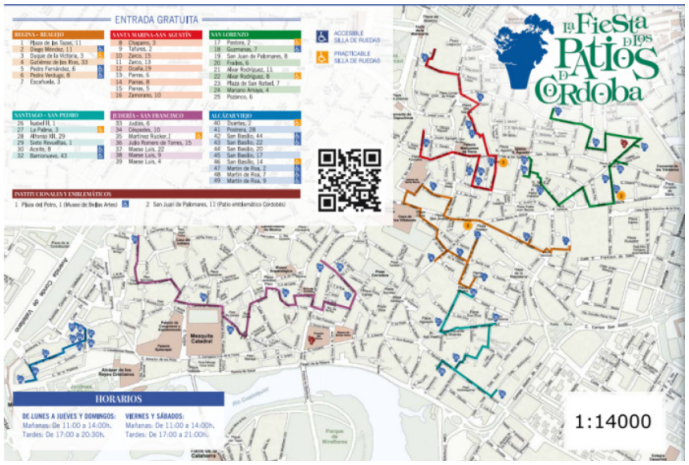


Fig. 1.1: El plano de la ciudad con los grafos a explorar por los drones.

vehículo base en general, sin un caso de estudio en particular, a través de Programación Lineal Entera Mixta. Es decir, planteamos un problema de optimización con el siguiente

formato:

$$\begin{aligned} &\text{mín } c^t x \\ &\text{sujeto a } Ax \geq b \\ &\qquad x_i \in \mathbb{R}, \forall i \in I \\ &\qquad x_j \in \mathbb{Z}, \forall j \in J \\ &\qquad x_k \in \{0,1\}, \forall k \in K \end{aligned}$$

El problema que estudiaremos es computacionalmente difícil, y normalmente se resuelven situaciones similares usando heurísticas. Como construiremos un modelo que lo resuelve de forma exacta, es de esperar que no se pueda encontrar la solución óptima en un tiempo razonable, incluso para instancias no muy grandes. No obstante, esperamos poder sacar conclusiones de los resultados parciales obtenidos.

2. ESTADO DEL ARTE DE LA OPTIMIZACIÓN CON DRONES

Nos dedicaremos a explorar la investigación existente en problemas de optimización de diferentes operaciones con drones, también llamados UAVs (Unmanned Aerial Vehicles), así como exponer algunos modelos y algoritmos que se han utilizado para encarar estos problemas. Mucha de la información descripta en esta sección se obtuvo de una investigación sobre el estado del arte publicada en 2018 [Otto et al., 2018].

2.1. Cobertura y monitoreo con UAVs

La cobertura y monitoreo con UAVs es útil en una gran variedad de contextos. Algunas aplicaciones estudiadas hasta ahora incluyen: monitoreo de áreas rurales, búsqueda de objetos, observación de áreas urbanas o con diversos obstáculos, respuesta frente a desastres como inundaciones o incendios y operaciones de delivery.

2.1.1. Problemas de cobertura total

En *problemas de cobertura total* se trabaja con un área, generalmente convexa, que debe ser observada en su totalidad con uno o varios UAVs. Muchas veces se considera que el dron tiene una huella (*footprint*): el área que el dron puede observar debajo de su ubicación; la huella y la resolución dependen de la distancia focal del dron y la altura a la que vuela. Éstas y otras variables son contempladas en Avellar et al. [Avellar et al., 2015], donde los autores consideran un área convexa y la cubren con un grafo basado en la reducción del tiempo de vuelo de un dron. Específicamente, consideran un UAV que vuela en *lawn mowing pattern*, también llamado *boustrophedon method*, cubriendo el ancho del área con un movimiento en zigzag. La dirección en la que el dron realiza el zig-zag es elegida para reducir su cantidad de giros. Luego se construye un grafo basado en ese recorrido individual y se plantea un problema de minimización del tiempo que se tarda en cubrir ese grafo con varios drones. Para esto los autores minimizan el tiempo de recorrido del dron que más tarda; luego quitan ese dron y las aristas que este cubrió y vuelven a minimizar el tiempo máximo entre los drones que quedan, y así siguiendo hasta completar todos los drones.
En otro trabajo, Barrientos et al. [Barrientos et al., 2011] proponen una subdivisión del área a cubrir en subáreas para visitar con varios drones, buscando minimizar el tiempo de cobertura. Para elegir las subáreas se realiza un reparto de tareas entre los drones, que quieren cubrir la mayor área posible cada uno, dentro de sus capacidades, minimizando la superposición de tareas.

En otros trabajos, la variable que se busca minimizar es la distancia recorrida. Por ejemplo, Dille y Singh [Dille and Singh, 2013] toman un grafo de calles que deben ser monitoreadas y proponen una teselación, cubriendo el grafo con órbitas circulares a partir de una evaluación de la forma y tamaño del área observada por un dron continuamente mientras vuela en círculo. Utilizan el modelo del *Dubins vehicle* [Dubins, 1957], pensado para ciertos tipos de drones que tienen dificultades para virar bruscamente y requieren un radio mínimo de giro. También detectan órbitas redundantes por estar totalmente cubiertas por otras y las

eliminan.

Otra variante también estudiada en la literatura es la minimización del costo energético total de la operación. Li, Wang y Sung [Li et al., 2016] modelan un área a cubrir con un mapa topológico de sus elevaciones y construyen la función a optimizar considerando variables que afectan al desempeño del dron, como el peso que lleva y la resistencia del viento. Amorosi et al. [Amorosi et al., 2018] abordan un problema de cobertura de Internet 5G mediante drones en un área rural, donde se considera que sólo algunas muy puntuales necesitan cobertura. En este caso existen áreas de guardado, áreas a cubrir con la señal de internet y arcos entre ellas con costos energéticos. También hay ventanas de tiempo en las cuales los drones pueden atravesar un arco o quedarse en un lugar. Se busca minimizar el costo energético total garantizando cobertura constante de 5G en las áreas que la requieren. Se considera que los UAVs deben tener siempre una cantidad mínima de energía disponible por posibles eventualidades. La propia distribución de áreas de guardado y de cobertura se optimizó en un trabajo anterior.

2.1.2. Problemas de cobertura parcial

También hay situaciones donde no se considera necesario cubrir con drones toda el área considerada. En ciertos casos, hay secciones cuya exploración se penaliza o directamente se prohíbe. Cakici et al. [Cakıcı et al., 2016] dividen el mapa en regiones deseadas y regiones prohibidas, y dentro de las primeras consideran algunas más valiosas que otras en términos de la información que contienen. Plantean un algoritmo genético para maximizar la información recolectada y llegar a un destino predeterminado. Ergezer y Leblebicioglu [Ergezer and Leblebicioglu, 2013] también trabajan con regiones deseadas y no deseadas, implementando un modelo que crea 'fuerzas repulsivas' como forma de penalizar ineficiencias para alejar a los drones de ciertos lugares y 'fuerzas atractivas' para acercarlos a otros. En [Gu et al., 2013] se proponen métodos para reducir los tiempos de recepción y procesamiento de información.

2.1.3. Operaciones de búsqueda

Se han planteado también situaciones donde los UAVs deben buscar un objeto dentro de un área. El objeto puede ser móvil, pero se suele suponer quieto por ser más lento que un dron. Lin y Goodrich [Lin and Goodrich, 2014] presentan el problema de buscar a una persona en un área con secciones que poseen diferentes características y obstáculos, de lo cual se desprende una probabilidad de encontrar a la persona en cada área en caso de que esté ahí. Se busca armar un camino de búsqueda que maximice la probabilidad de encontrarla. Ji et al. [Ji et al., 2015] construyen una búsqueda cooperativa entre varios drones en un área no convexa, donde modelan la probabilidad de que un dron encuentre un objeto según el ángulo donde se encuentra respecto al campo de visión del dron (si está más al borde, se considera más difícil encontrarlo).

2.2. Operaciones combinadas entre drones y vehículos base

Pasamos a dar detalles sobre trabajos más vinculados con lo que abordamos en esta tesis. El uso de drones en problemas de ruteo suele asociarse con vehículos complementarios, que sirven para ejecutar una operación en paralelo o bien como vehículos base, a donde

los drones pueden volver y recargarse. En estos casos se presume que la autonomía de un dron es corta, y por eso necesita volver a una base móvil que se presume tiene autonomía extensa o incluso no acotada (no necesita ayuda externa), como puede ser un camión o una embarcación. En casi todos los trabajos que hemos encontrado donde uno o varios drones retornan a un vehículo base, el mismo es uno solo. El vehículo base se acerca a lugares adecuados para que los drones vuelvan antes de que su autonomía se termine. En otros casos no hay vehículo base, sino *warehouses*, lugares de partida y retorno fijos, que pueden ser uno o varios distribuidos por el área donde se ejecuta la operación.

2.2.1. Problemas clásicos relacionados

El *Traveling Salesman Problem* (TSP) es un famoso problema de optimización combinatoria donde se tiene un vehículo o individuo que debe viajar por un grafo completo con costos en las aristas, pasando exactamente una vez por cada nodo y regresando al punto de partida de forma que minimice el costo total. Una generalización clásica del TSP es el *Vehicle Routing Problem* (VRP), donde se plantea el mismo problema pero utilizando una flota de vehículos. En 1959 lo plantean por primera vez Dantzig y Ramser como *Truck Dispatching Problem* [Dantzig and Ramser, 1959]. Furini, Persiani y Toth [Furini et al., 2016] publicaron en 2016 una aplicación para drones en espacio aéreo controlado del *Time Dependent Traveling Salesman Problem*, donde el costo de atravesar una arista depende del momento en el cual se la atraviese. Estos problemas son NP-hard. Por eso, además de métodos exactos con programación lineal entera mixta, se han planteado diferentes heurísticas. Encontramos varias referencias recurriendo a la investigación de estado del arte publicada por Liu et al. [Liu et al.,]. Clarke y Wright [Clarke and Wright, 1964] introducen en 1964 el *savings algorithm*, una heurística que aún hoy se utiliza. Entre las heurísticas constructivas, que arman posibles soluciones desde cero, podemos contar el *Nearest Neighbor Method*, un algoritmo goloso donde se van agregando progresivamente las aristas de menor costo. Se ha utilizado en sí mismo como método de resolución [Joshi and Kaur, 2015], pero en los últimos años es más común complementarlo con otros métodos, usándolo a veces para hallar una solución inicial. También se han utilizado métodos de mejora de soluciones parciales, como *Intra-route method* e *Inter-Route method*. Se han usado para el VRP metaheurísticas clásicas como *Simulated Annealing* y *Tabu Search* [Osman, 1993], *Iterated Local Search* [Tang and Wang, 2006], algoritmos genéticos [Hwang, 2002] y algoritmos de colonia de hormigas [Bell and McMullen, 2004].

Todos los problems mencionados arriba involucran el requerimiento de visitar los vértices de un grafo. Sin embargo, en algunos casos el interés puede centrarse en recorrer las aristas/arcos. Aparece por lo tanto el *Arc Routing Problem* (ARP), consistente en buscar el costo mínimo para explorar aristas de un grafo. Para investigar el estado del arte de los ARPs tomamos como referencia un trabajo recopilatorio publicado en 2021 [Corberán et al., 2021]. Este problema tiene diferentes variantes; una de las más antiguas es el *Chinese Postman Problem* (CPP), introducido por primera vez por Guan en 1962 [Mei-Ko, 1962]. Plantea la minimización del costo de recorrer todos los arcos de un grafo por lo menos una vez por un servidor (cartero). Este problema fue resuelto en tiempo polinomial en 1973 por Edmonds y Johnson [Edmonds, 1973]. Del CPP se deriva el *Rural Postman Problem* (RPP), donde ya no se requiere recorrer todos los arcos sino sólo algunos de los que están disponibles. Lo introduce Orloff en 1974 [Orloff, 1974], y más adelante se demuestra que es NP-hard. Goldman y Wong [Golden and Wong, 1981] introducen en

1981 el *Capacitated Arc Routing Problem* (CARP), donde los arcos a servir tienen asociada una demanda y los vehículos una capacidad limitada. En estos problemas, los arcos pueden ser direccionados, no direccionados o de ambos tipos en un mismo grafo. Dentro de esta última variante, en 2010 Gouveia, Mourâo y Pinto [Gouveia et al., 2010] implementan un modelo MILP para que un vehículo complete la demanda de los arcos que la tienen en uno o varios viajes. La conexión de los caminos se garantiza mediante el uso de restricciones de flujo. Más adelante en la tesis nos inspiramos en este trabajo para probar conexión de caminos en el modelo que desarrollamos. También existen los problemas *windy*, donde el costo de atravesar un arco varía según el sentido. Otros ARPs involucran situaciones dinámicas, donde el mapa a servir puede cambiar con el tiempo, o ventanas temporales.

Una variante de interés para este trabajo son los problemas de Arc Routing con drones. Estos traen un cambio importante respecto a los otros que mencionamos porque en aquellos no importa la forma del arco sino sólo el costo de atravesarlo, dado que este debe ser recorrido en su totalidad o para nada. Un dron puede entrar y salir de un arco por cualquier punto, lo que en muchos casos genera problemas de optimización con conjuntos no numerables de soluciones factibles. El Drone RPP es introducido por primera vez por Campbell et al. [Campbell et al., 2018], quienes proponen en 2018 un algoritmo exacto para resolver diferentes casos. Toman el caso donde el dron tiene capacidad ilimitada, por lo cual alcanza con uno solo, como también casos con capacidad limitada. En estas situaciones, permiten que el servicio a un arco sea compartido por varios drones. Atienden la limitación de que el problema no es discreto aproximando los arcos con poligonales.

2.2.2. Modelos con vehículos complementarios y bases de recarga

En estos problemas aparecen diferentes tipos de vehículos, incluyendo drones, que deben coordinarse de alguna manera para ejecutar una operación. Igual que en otros tipos de operaciones que involucran drones, las variables a minimizar pueden ser el tiempo, la distancia total o el costo energético; en algunos casos se busca minimizar la cantidad de drones necesarios. Viguria, Maza y Ollero [Viguria et al., 2010] presentan en 2009 una aplicación del *Multi Robot Task Allocation Problem* (MRTA) para detectar y extinguir incendios. Desarrollan un nuevo algoritmo market-based que llaman $S + T$ para repartir un conjunto de tareas entre robots y ejecutarlas de forma óptima. Sujit, Sousa y Pereira [Sujit et al., 2009] modelan una operación donde un UAV coordina a varios vehículos autónomos acuáticos en una exploración marítima. El dron les transmite a los AUV por dónde explorar y recibe la información que consiguen, para luego enviarles la señal de cómo continuar; se tiene en cuenta que los AUV tienen un rango de comunicación limitado. El área marítima a explorar se discretiza dividiéndola en celdas.

Una situación de interés para este trabajo es aquella donde se cuenta con drones para realizar una tarea y vehículos base que les sirven de apoyo (como punto de recarga o simplemente para viajar a otro punto sin gastar energía). Garone et al. [Garone et al., 2010] publican un trabajo donde se buscan formas óptimas de que un vehículo rápido con autonomía corta visite un conjunto de puntos, ayudado por un vehículo lento de autonomía extensa que puede llevarlo a puntos convenientes de despegue. En otro trabajo, publicado por Luo, Liu y Shi [Luo et al., 2017], un vehículo base lleva un dron por un grafo hasta diferentes estacionamientos, que también son los puntos donde puede recogerlo, mientras el dron recorre otras aristas inaccesibles al vehículo base hasta nodos objetivo. Se abor-

da este problema con un modelo de programación entera binaria, y se diseña y compara experimentalmente dos heurísticas. Mathew y Smith presentan una aplicación a delivery de un problema similar y muestran que es NP-hard reduciéndolo al *Generalized Traveling Salesman Problem*. Aprovechan este hecho para utilizar heurísticas ya conocidas para este último problema. Amorosi, Puerto y Valverde [Amorosi et al., 2021] abordan el problema de llevar un dron mediante un vehículo base, moviéndose libremente por el plano, a explorar diferentes grafos, donde se considera suficiente recorrer un cierto porcentaje de la suma de longitudes de sus aristas. Más adelante extienden el problema a una versión donde se cuenta con una base y varios drones [Amorosi et al., 2023]. Yakici et al. [Yakıcı et al., 2024] publicaron en 2024 un trabajo para aplicación bélica, donde varios vehículos de carga se mueven y lanzan drones para cazar objetivos móviles, cuya ubicación se representa con nodos dependientes del tiempo. Los drones pueden volver a una base distinta de aquella de la que despegaron. En el paper se determina una solución exacta con programación lineal entera y, para obtener una solución más rápida, utilizan una heurística de colonia de hormigas.

3. CONSTRUCCIÓN DEL MODELO DISCRETO

3.1. Descripción del problema

En el capítulo de investigación sobre literatura de optimización con drones mencionamos un paper de Amorosi, Puerto y Valverde del año 2023 [Amorosi et al., 2023] donde se plantea que varios drones deben monitorear ciertas zonas, y hay un vehículo base que los lleva a puntos de partida y llega a buscarlos a puntos de retorno para recargarlos. Las zonas a monitorear se encuentran en el plano y están representadas por grafos, donde lo que se debe recorrer con drones son las aristas de estos grafos. Más específicamente, sólo es necesario recorrer ciertos porcentajes de la longitud total de cada arista; o bien porcentajes de la suma de longitudes de las aristas de cada grafo. Se presume que la autonomía del vehículo base es extensa y la de los drones es corta, por lo que estos deben volver a la base antes de que su energía se agote. También se supone que tanto los drones como el vehículo base pueden moverse libremente en el plano, así que las distancias entre puntos son euclídeas. En el paper en cuestión se diseña un modelo de optimización convexa para minimizar la distancia total que recorre el vehículo base a lo largo de toda la operación; también se les da a los drones la posibilidad de ingresar a las aristas de los grafos y salir de ellas por cualquier punto, siempre y cuando se cumpla el porcentaje mínimo de recorrido. De esta forma, se tiene un problema continuo, con un espacio no numerable de soluciones factibles.

En este trabajo construimos una variante discretizada para convertir el problema en uno de optimización combinatoria. Lo resolvemos con un modelo de programación lineal entera mixta basado en las restricciones y la función objetivo del paper mencionado [Amorosi et al., 2023]. Dentro del plano donde trabajamos, construimos un grafo conexo, cuyos nodos y aristas conformarán los caminos bidireccionales que puede recorrer el vehículo base. Lo llamaremos 'grilla' para distinguirlo de los grafos a recorrer por los drones, a los cuales suponemos conexos. Las aristas de los grafos deberán ser recorridas en su totalidad por drones (al menos en un sentido), y además un dron podrá viajar de cualquier nodo de un grafo a cualquier otro del mismo grafo; es decir, los grafos se completarán; estas aristas nuevas serán de uso opcional para completar el recorrido de las obligatorias. Estos nuevos grafos completos pasarán a ser digrafos, con todos los arcos en ambos sentidos, para simbolizar el hecho de que se pueden recorrer en forma bidireccional. En resumen, realizamos un intercambio entre complejidad computacional (más variables y restricciones en nuestro modelo) y simplicidad y facilidad de resolución del tipo de modelo (de programación convexa a lineal).

El proceso de discretización es el siguiente. En primer lugar, tenemos una zona rectangular del plano que contiene a los grafos que se deben explorar, como se ve en la Figura 3.1. Sobre esta zona construimos la grilla con filas y columnas, como se ve en la Figura 3.2. Definimos que las aristas de la grilla serán las únicas que puede recorrer el vehículo base. Todas las aristas del modelo se podrán recorrer en ambos sentidos. Se muestra un ejemplo de grilla con filas y columnas equidistantes, pero esto no es un requisito para el modelo. Tampoco se requiere que los arcos de la grilla sean verticales y horizontales, aunque en esta tesis lo hacemos de esta manera. Implementamos y evaluamos dos discretizaciones

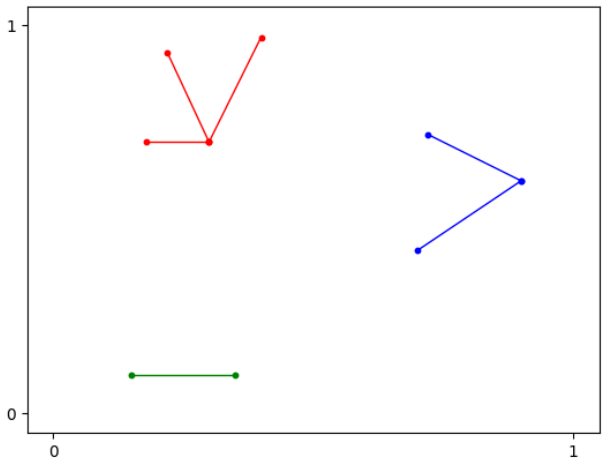


Fig. 3.1: Tres grafos ubicados sobre el plano, cada uno con un color.

alternativas para la grilla. En la primera opción, el vehículo sólo contará con las aristas entre nodos vecinos de acuerdo a la Figura 3.2, con costo de recorrido igual a la longitud euclídea. En la segunda, los nodos de la grilla formarán un grafo completo, pero el costo de atravesar una arista puede variar. Será la distancia Manhattan, que llamaremos distancia 1, si en la situación representada la base debería viajar por aristas de la grilla (por ejemplo, un vehículo terrestre que debe ir por calles de una ciudad en damero). La distancia 1 está dada por $d_1(x, y) = |x_1 - y_1| + |x_2 - y_2|$ para $x, y \in \mathbb{R}^2$. En cambio, se usará distancia euclídea, que llamaremos distancia 2, si la base no tiene restricciones de por dónde puede moverse (por ejemplo, si es un helicóptero o una embarcación). Llamaremos 'grilla damero' a la primera alternativa y 'grilla completa' a la segunda. Más específicamente, hablaremos de grilla completa 1 o grilla completa 2 según la métrica usada para representar las distancias entre nodos.

Cuando usamos la distancia Manhattan asumimos implícitamente que todas las 'calles' están disponibles en ambos sentidos, y este será el caso a estudiar en este trabajo. Se pueden implementar las distancias de forma más acorde a una ciudad real, incluso una con obstáculos imprevistos, calculando el camino mínimo entre cada par de nodos de la grilla, por ejemplo utilizando el algoritmo de Dikjstra. Esto se aplicaría al grafo formado por las aristas verticales y horizontales en los sentidos disponibles con pesos iguales a sus longitudes euclídeas.

Cada nodo de cada grafo estará conectado por aristas con todos los nodos de la grilla. Los drones sólo podrán recorrer las aristas entre un nodo de la grilla y uno de grafo, y las aristas de cualquier nodo a cualquier otro dentro de un mismo grafo. Para esto completamos los grafos con aristas bidireccionales de recorrido no obligatorio (Figura 3.3). En todos los arcos disponibles para el dron, el costo de atravesarlos será la distancia euclídea entre los extremos, dado que un dron puede volar por cualquier segmento en el plano.

La base viaja a diferentes nodos de la grilla donde lanza y recoge los drones, teniendo en cuenta que éstos deben llegar al punto de encuentro con la base antes de que su autonomía se acabe. El viaje de la base se divide en *etapas*, empezando por la etapa 1. En las

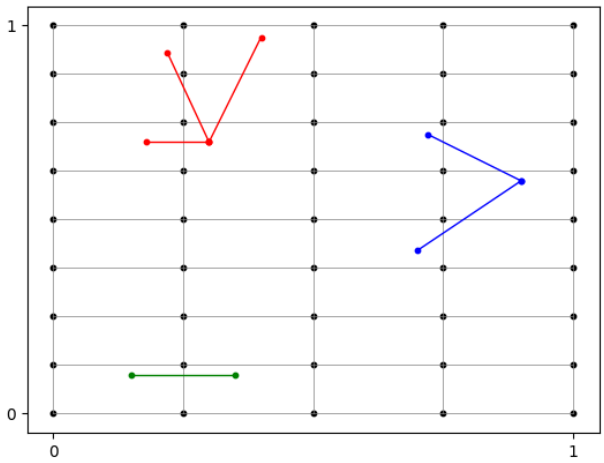


Fig. 3.2: Los grafos ubicados sobre la grilla.

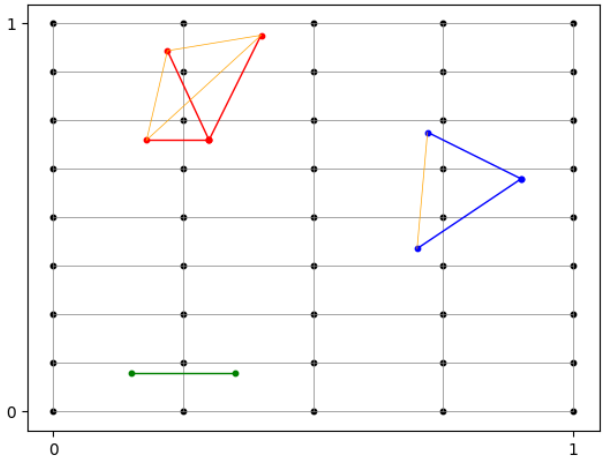


Fig. 3.3: Los grafos no completos se completan en anaranjado.

etapas impares, la base viaja hacia un punto desde donde lanzará uno o más drones. En las etapas pares, la base lanza los drones desde el punto de inicio y viaja hacia el punto en donde los recogerá. Este último es el punto de partida de la siguiente etapa. En cada etapa, la base debe viajar desde el nodo de partida hasta el de retorno. Para la grilla damero, puede ser necesario recorrer varias aristas verticales u horizontales. Para la grilla completa, basta con recorrer la arista que conecta la partida con el retorno. Ilustramos esto en la Figura 3.4. Para viajar entre los dos puntos de la grilla que queremos unir, la grilla damero activa 5 variables asociadas a las líneas punteadas marrones. En cambio, en la grilla completa se activa sólo una variable, representada por la línea violeta. En ambos casos el costo total es el mismo, dado por la distancia 1.

La cantidad de etapas es el doble de la de grafos a explorar porque en cada una de las etapas pares se debe explorar al menos un grafo hasta que ya se hayan recorrido todos, por lo que hacen falta a lo sumo tantas etapas pares como grafos. El dron que sale a explorar un grafo irá directo del nodo de partida a un nodo del grafo a explorar; y una

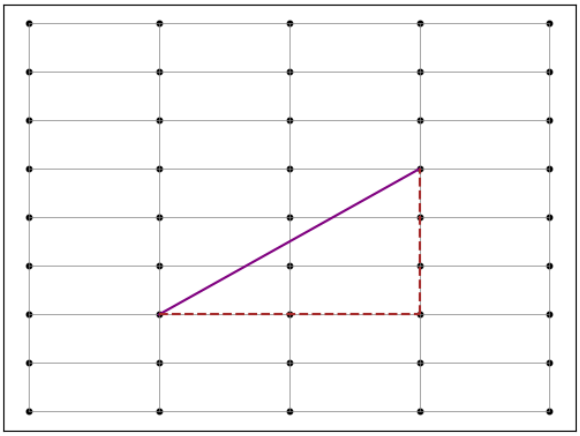


Fig. 3.4: Viaje entre dos nodos en grilla damero y grilla completa.

vez terminada la exploración del grafo irá del último nodo que visitó al nodo de retorno. El dron no podrá usar otras conexiones de grafo a grilla además de la que usa para entrar y la que usa para salir. Existen nodos distinguidos de la grilla, denominados *orig* y *dest* que son el punto de salida y el de llegada del vehículo base respectivamente (pueden ser diferentes o el mismo). El objetivo del problema que formulamos es encontrar un recorrido de la base y recorridos de los drones tales que se explore todos los grafos de forma que minimice la duración total de la operación. En cambio, en el trabajo que usamos como referencia se minimiza el tiempo total que viaja la base, incluyendo la imposición de que los drones lleguen al punto de retorno antes que la base en cada etapa.

3.2. Formulación matemática

Comenzamos presentando los parámetros y los conjuntos que van a estar dados en el problema que queremos resolver. Cada grafo que debe ser explorado se convierte en un digrafo, con cada arista reemplazada por dos arcos entre los mismos nodos en sentidos opuestos.

Nombre	Definición
$\mathcal{D}$	conjunto de drones
$\mathcal{K}$	conjunto de nodos
$\mathcal{G}$	conjunto de grafos a explorar
$\mathcal{K}_g$	$\subset \mathcal{K}$, conjunto de nodos del grafo $g \in \mathcal{G}$
$\mathcal{K}_{grid}$	$\subset \mathcal{K}$, conjunto de nodos de la grilla
τ	conjunto de etapas, $\|\tau\| = 2\|\mathcal{G}\|$
τ_P	conjunto de etapas pares (en las que se puede lanzar drones)
τ_I	conjunto de etapas impares (en las que no se puede lanzar drones)
V_v	conjunto de vecinos del nodo v (tanto del grafo como de la grilla)
V_v^{grid}	conjunto de vecinos de un nodo $v \in \mathcal{K}_{grid}$ en $\mathcal{K}_{grid}$
$D_1(v, w)$	distancia Manhattan entre los nodos $v, w \in \mathcal{K}_{grid}$
$D_2(v, w)$	distancia euclídea entre los nodos $v, w \in \mathcal{K}$
v_M	velocidad del vehículo base
v_D	velocidad de los drones
$N_{\mathcal{D}}$	tiempo de autonomía de un dron
E_g	conjunto de aristas direccionadas del digrafo g completado
$\tilde{E}_g$	conjunto de aristas no dirigidas obligatorias del grafo g original
E_{grid}	conjunto de aristas direccionadas de la grilla

Seguimos con las variables que va a tener nuestro modelo discreto, ya sea en el caso de grilla damero o grilla completa.

Variable	Definición
$z_{v,w}^{d,t}$	$\forall (v, w) \in E \setminus E_{grid}$; =1 si el dron d va de v a w en la etapa $t \in \tau_P$, 0 si no
$m_{v,w}^t$	$\forall v, w \in \mathcal{K}_{grid} : w \in V_v$; = 1 si la base va de v a w en la etapa t, 0 si no
L_v^t	$\forall v \in \mathcal{K}_{grid}$; = 1 si en el nodo v comienza la etapa t, 0 si no
R_v^t	$\forall v \in \mathcal{K}_{grid}$; = 1 si en el nodo v termina la etapa t, 0 si no
$\mu_g^{d,t}$	$\forall g \in \mathcal{G}, d \in \mathcal{D}, t \in \tau_P$; = 1 si el dron d explora el grafo g en etapa t, 0 si no
β_t	$\forall t \in \tau$; = 1 si al comenzar la etapa t todos los grafos fueron explorados, 0 si no
p_v^t	$\in \mathbb{Z}, \forall v \in \mathcal{K}_{grid}, t \in \tau$; orden de v para la base en etapa t (sólo grilla damero)
k_t	$\in \mathbb{Z}, \forall t \in \tau$; ≥ 0; cantidad de grafos explorados durante la etapa t
h_t	$\in \mathbb{R}, \forall t \in \tau$; cota superior de la duración de la etapa t

3.2.1. Restricciones elementales

Introducimos las restricciones correspondientes a los puntos de partida y de retorno. Para empezar, hay uno y sólo uno de cada tipo por etapa:

$$\forall t \in \tau : \sum_{v \in \mathcal{K}_{grid}} L_v^t = 1 \tag{3.1}$$

$$\forall t \in \tau : \sum_{v \in \mathcal{K}_{grid}} R_v^t = 1 \tag{3.2}$$

La trayectoria de la base empieza en *orig* y termina en *dest*:

$$L_{orig}^1 = 1 \tag{3.3}$$

$$R_{dest}^{2|\mathcal{G}|} = 1 \tag{3.4}$$

Cada punto de terminación de una etapa es el de comienzo de la siguiente. Esto garantiza que el orden numérico de etapas corresponda realmente a un orden cronológico, evitando que haya una cantidad exponencial de soluciones equivalentes.

$$\forall 1 \leq t < 2|\mathcal{G}|, v \in \mathcal{K}_{grid} : R_v^t = L_v^{t+1} \tag{3.5}$$

Queremos que los caminos de los drones estén bien definidos, por lo que vamos a imponer que los drones entren y salgan la misma cantidad de veces de cada nodo del grafo que recorren.

$$\forall g \in \mathcal{G}, t \in \tau_P, d \in \mathcal{D}, v \in \mathcal{K}_g : \sum_{w \in V_v} (z_{v,w}^{d,t} - z_{w,v}^{d,t}) = 0 \tag{3.6}$$

La buena definición del camino en cada etapa también es requerida para el vehículo base, y para eso cada nodo debe tener la misma cantidad de ingresos que de salidas, excepto quizá el inicio y el retorno. El conjunto V_v^{grid} varía dependiendo del modelo. Si es la grilla damero, son los nodos que están inmediatamente arriba, abajo, a la izquierda y a la derecha de v. Si es la grilla completa, son todos los demás nodos de la grilla.

$$\forall t \in \tau, v \in \mathcal{K}_{grid} : \sum_{w \in V_v^{grid}} (m_{v,w}^t - m_{w,v}^t) = L_v^t - R_v^t \tag{3.7}$$

Ahora damos las restricciones asociadas a los grafos que debemos explorar. Primero indicamos que toda arista de recorrido obligatorio debe ser atravesada en algún sentido por algún dron:

$$\forall g \in \mathcal{G}, \{v, w\} \in \tilde{E}_g : \sum_{t \in \tau_P} \sum_{d \in \mathcal{D}} (z_{v,w}^{d,t} + z_{w,v}^{d,t}) \geq 1 \tag{3.8}$$

Procedemos a definir una variable que indica si un grafo fue explorado por un dron en una etapa. Recordamos que las etapas impares están establecidas como viajes de la base llevando los drones, por lo que estos no actúan:

$$\forall d \in \mathcal{D}, t \in \tau_P, g \in \mathcal{G} : \mu_g^{d,t} \geq \frac{1}{|E_g|} \sum_{(v,w) \in E_g} z_{v,w}^{d,t} \tag{3.9}$$

$$\forall d \in \mathcal{D}, t \in \tau_P, g \in \mathcal{G} : \mu_g^{d,t} \leq \sum_{(v,w) \in E_g} z_{v,w}^{d,t} \tag{3.10}$$

$$\sum_{t \in \tau_I} \sum_{d \in \mathcal{D}} \sum_{g \in \mathcal{G}} \mu_g^{d,t} = 0 \tag{3.11}$$

Ahora determinamos que cada grafo de $\mathcal{G}$ tiene que ser explorado exactamente por un dron en una etapa:

$$\forall g \in \mathcal{G} : \sum_{t \in \tau} \sum_{d \in \mathcal{D}} \mu_g^{d,t} = 1 \tag{3.12}$$

Además, cada dron que sale en una etapa explorará exactamente un grafo. De esta forma reducimos el espacio de soluciones y evitamos despegues de drones que no ayuden a cumplir el objetivo de exploración.

$$\forall t \in \tau_P, d \in \mathcal{D} : \sum_{g \in \mathcal{G}} \mu_g^{d,t} \leq 1 \tag{3.13}$$

En todas las etapas, los drones lanzados deben llegar al punto de retorno antes de que su autonomía se acabe. También definimos una cota superior para el tiempo que tardan la base y los drones en completar la etapa, a fin de darles margen para que no lleguen al mismo tiempo.

$$\forall t \in \tau : \frac{1}{v_M} \sum_{(v,w)\in E_{grid}} D_i(v,w) m^t_{v,w} \leq h_t \tag{3.14}$$

$$\forall d \in \mathcal{D}, t \in \tau_P : \frac{1}{v_D} \sum_{(v,w)\in E\setminus E_{grid}} D_2(v,w) z^{d,t}_{v,w} \leq N_{\mathcal{D}} \tag{3.15}$$

$$\forall d \in \mathcal{D}, t \in \tau_P : \frac{1}{v_D} \sum_{(v,w)\in E\setminus E_{grid}} D_2(v,w) z^{d,t}_{v,w} \leq h_t \tag{3.16}$$

$i \in \{1,2\}$ indica qué distancia se usa para medir la longitud de las aristas que recorre la base, ya que puede cambiar dependiendo de si trabajamos con la grilla damero o con la grilla completa 1 o grilla completa 2.

3.2.2. Restricciones de conectividad para los drones

Necesitamos asegurarnos de que no haya subtours, es decir que los caminos de la base y de los drones sean continuos y no tengan ciclos aislados. Comenzamos presentando SEC (Subtour Elimination Constraints) para los drones basadas en el paper de referencia [Amorosi et al., 2023], que además son clásicas en la literatura. Sin embargo, como explicaremos, no se pueden utilizar en nuestro caso. Primero se optimiza sin ellas, y si aparece un subtour se agrega al modelo una restricción para eliminarlo. Notamos que sólo se necesita eliminar ciclos desconectados de la partida o la llegada de la etapa; un camino válido podría contener un ciclo.

$$\forall t \in \tau_P, d \in \mathcal{D} : \sum_{\{v,w\}:v,w\in S} z^{d,t}_{v,w} \leq |S| - 1, \forall S \subset \mathcal{K}$$

No obstante, estas SEC no son válidas en nuestro problema para los recorridos de los drones por los grafos porque, si cortamos un ciclo, podríamos perder una trayectoria válida del dron que contenga ese ciclo. Además, desarmar un ciclo de j nodos podría simplemente producir un ciclo nuevo con sólo un nodo de diferencia, entre todos los nodos del grafo o de la grilla, para ciclos del vehículo base. Aunque en muchos contextos esto no es un problema, por ejemplo en el TSP donde agregar pocas de estas SEC suele ser suficiente, en este caso no necesitamos minimizar la trayectoria del dron. De ahí que podrían aparecer muchos ciclos nuevos a medida que vamos cortándolos. En cambio, sí es válido eliminar ciclos en el camino de la base, dado que si un camino válido de la base en una etapa contiene un ciclo se puede obtener un camino válido más corto, tomando el mismo camino excepto por la eliminación de este ciclo.

Establecemos que el dron que recorre un grafo en una etapa sólo podrá usar en esta etapa una arista de la grilla al grafo y una del grafo a la grilla.

$$\forall g \in \mathcal{G}, t \in \tau_P, d \in \mathcal{D} : \sum_{v\in\mathcal{K}_{grid}, w\in K_g} z^{d,t}_{v,w} = \mu^{d,t}_g \tag{3.17}$$

$$\forall g \in \mathcal{G}, t \in \tau_P, d \in \mathcal{D} : \sum_{v \in \mathcal{K}_{grid}, w \in K_g} z_{w,v}^{d,t} = \mu_g^{d,t} \tag{3.18}$$

Más aún, el nodo del que el dron salga hacia el grafo debe ser el de inicio de la etapa, y el nodo al que llega después debe ser el de retorno. De esta manera, representamos bien el hecho de que los drones pueden volar desde cualquier punto hasta cualquier otro, y por lo tanto llegarán al nodo inicial del grafo a recorrer por el camino más corto desde el lugar donde despegaron del vehículo base. De la misma forma, irán desde el nodo donde terminaron de recorrer el grafo directamente hacia el punto donde ha de recogerlos el vehículo base.

$$\forall v \in \mathcal{K}_{grid}, t \in \tau_P, d \in \mathcal{D}, g \in \mathcal{G} : \sum_{w \in \mathcal{K}_g} z_{v,w}^{d,t} \leq L_v^t \tag{3.19}$$

$$\forall v \in \mathcal{K}_{grid}, t \in \tau_P, d \in \mathcal{D}, g \in \mathcal{G} : \sum_{w \in \mathcal{K}_g} z_{w,v}^{d,t} \leq R_v^t \tag{3.20}$$

Procedemos a demostrar que en toda solución factible los caminos de los drones son conexos, es decir, no contienen subtours, a partir de la suposición con que armamos nuestro modelo de que las aristas no direccionadas de recorrido obligatorio forman grafos conexos.

Proposición 3.2.1. *Dada una solución factible de nuestro modelo, sean $d \in \mathcal{D}, t \in \tau_P, g \in \mathcal{G}$ tales que $\mu_g^{d,t} = 1$. Entonces, los arcos $(v, w) \in E \setminus E_{grid}$ tales que $z_{v,w}^{d,t} = 1$ forman un camino conexo.*

Demostración. Sean $v_1, v_2 \in \mathcal{K}_g$ los nodos de inicio y finalización del recorrido del grafo respectivamente. Supongamos que existe un subtour, es decir, un ciclo de arcos recorridos por el dron d en etapa t desconectado de v_1 o de v_2. Como el grafo original no dirigido de exploración obligatoria era conexo, existe un camino de aristas obligatorias entre v_1 y el subtour, y también entre v_2 y el subtour. De estos caminos obligatorios algunos arcos no se recorren ya que el subtour estaba aislado de v_1 o de v_2 por hipótesis. Además, estos arcos obligatorios no pueden ser recorridos por otro dron ni en otra etapa por (3.9) y (3.12), dado que el subtour fue activado por el dron d en etapa t. Pero esto contradice (3.8), restricción que impone recorrer al menos una vez todos los arcos obligatorios. Por lo tanto, no hay subtours. □

3.2.3. Conectividad para el vehículo base: grilla damero o completa

En esta sección vamos a dar un conjunto de restricciones para el caso donde implementamos la grilla damero, y luego otro para la grilla completa. Los dos modelos usan todas las restricciones numeradas de las otras secciones. La grilla damero tiene más restricciones, mientras que la grilla completa tiene más variables porque incluye arcos entre todos los pares de nodos para todas las etapas y no sólo entre los contiguos, mientras que la grilla damero sólo suma tantas variables como nodos de la grilla por cada etapa. Más adelante realizaremos una comparación experimental del desempeño entre las dos posibilidades. Empezamos por el primer caso.

De (3.7) se sigue directamente que todo camino factible que no contenga un ciclo comienza en $v \in \mathcal{K}_{grid}$ tal que $L_v^t = 1$ y termina en $w \in \mathcal{K}_{grid}$ tal que $R_w^t = 1$.

Como buscamos minimizar la trayectoria de la base, todo camino que incluya ir del nodo

de partida al de retorno y además tenga más arcos (por ejemplo, que tenga un ciclo) se considera que tiene 'redundancias' porque gasta tiempo de más en volver al punto de partida o en dirigirse otra vez al punto de retorno una vez alcanzado. Si quisiéramos eliminar estas redundancias podríamos agregar la siguiente desigualdad, pero no es necesario dado que toda solución óptima la verifica o puede ser cambiada por una solución óptima que la verifica.

$$\forall v \in \mathcal{K}_{grid}, t \in \tau : \sum_{w \in V_v^{grid}} (m_{v,w}^t + m_{w,v}^t) \leq 2 - L_v^t - R_v^t$$

Primero notamos que si una etapa tiene el inicio y el retorno en el mismo punto, esta desigualdad indica que no se entra ni se sale de él. En otras palabras, la trayectoria en esta etapa sería nula (luego mostraremos que no hay subtours externos a este punto). Si hubiera un ciclo que pasara por este punto, toda cota superior h_t de la longitud de este ciclo sería no negativa, y por lo tanto cota superior de la longitud 0 del camino nulo. Otras h_k con $k \neq t$ no se ven afectadas. Luego, podemos asumir que una solución óptima verifica esta desigualdad para una etapa que comienza y termina en el mismo lugar.

En caso de que el nodo de partida y el de retorno sean diferentes, la desigualdad afirma que no se vuelve a entrar al nodo de partida v (del que se sale por (3.7)) ni se vuelve a salir del de llegada w (al que se llega por la misma restricción). En particular, ningún ciclo pasa por v o w. Sin esta desigualdad, supongamos que un camino que pasa por v y w entra a v. Si pasa por w antes de entrar a v, se puede eliminar el camino posterior a entrar a w. Si el camino original pasa por w luego de entrar a v, quitamos el camino anterior a entrar a v. En cualquier caso, los h_t factibles para el camino más largo lo son para el más corto, así que no se pierde optimalidad. Análogamente, un camino que pase dos veces por w también puede ser acortado sin perder optimalidad. En conclusión, podemos asumir que en una solución óptima no hay ciclos que pasen por el inicio o el retorno de ninguna etapa.

Pero todavía no vimos qué pasa si hay ciclos que no pasen por v ni por w (los cuales serían subtours), mientras el camino entre y salga una vez de cada nodo. Para eliminarlos, vamos a presentar e implementar restricciones de conectividad tipo MTZ [Miller et al., 1960], donde definiremos orden de recorrido de los nodos de la grilla. Este orden será menor o igual al total de aristas activadas por la base durante la etapa. Presentamos las variables p_v^t, que representarán este orden de recorrido de los nodos.

$$\forall v \in \mathcal{K}_{grid}, \forall t \in \tau : p_v^t \geq 0 \tag{3.21}$$

$$\forall v \in \mathcal{K}_{grid}, t \in \tau : p_v^t \leq \sum_{(w,q) \in E_{grid}} m_{w,q}^t \tag{3.22}$$

$$\forall (v,w) \in E_{grid}, t \in \tau : p_v^t - p_w^t + |\mathcal{K}_{grid}| m_{v,w}^t \leq |\mathcal{K}_{grid}| - 1 \tag{3.23}$$

Veamos ahora que estas restricciones garantizan ausencia de subtours.

Proposición 3.2.2. *Dadas una solución factible para el modelo de grilla damero y una etapa $t \in \tau$, sean $v, w \in \mathcal{K}_{grid}$ tales que $L_v^t = R_w^t = 1$. Con los arcos activados por la base (los $(v,w) \in E_{grid}$ tales que $m_{v,w}^t = 1$) no se puede formar un ciclo que no pase por v ni por w.*

Demostración. Supongamos que se forma un ciclo $v_1, v_2, \ldots, v_k, v_1$. De (3.23) se sigue que $p^t_{v_1} < p^t_{v_2} < \ldots < p^t_{v_k} < p^t_{v_1}$, y esto es un absurdo porque asumimos que se tenía una solución factible. □

Falta ver que los caminos que deberían ser factibles (sin ciclos, de la partida al retorno) efectivamente lo son.

Proposición 3.2.3. *Para una etapa $t \in \tau$, sean $v, w \in \mathcal{K}_{grid}$ tales que $L^t_v = R^t_w = 1$. Si $v = w$, el camino nulo de la base es factible en el modelo de grilla damero. Si $v \neq w$, todo camino sin ciclos que comienza en v y termina en w es factible.*

Demostración. Comenzamos por el camino nulo. Por (3.21) y (3.22) queda $p^t_v = 0$ para todo $v \in \mathcal{K}_{grid}$. Como todos los $m^t_{v,w}$ son 0, (3.23) es trivialmente verdadera. Lo mismo para (3.7).
Si $v_1 \neq v_2$, supongamos que los arcos $(v, w) \in E_{grid}$ tales que $m_{v,w} = 1$ forman un camino sin ciclos de v_1 a v_2. Primero observamos que este camino verifica (3.7). A continuación vemos que $\sum_{(v,w)\in E_{grid}} m^t_{v,w} < |\mathcal{K}_{grid}|$, porque lo contrario indicaría que se produce por lo menos un ciclo. Como suponemos que no hay ciclos, esto no ocurre. Entonces, dados $v, w \in \mathcal{K}_{grid}$ resulta $p^t_v - p^t_w \leq |\mathcal{K}_{grid}| - 1$ por (3.21) y (3.22). Si $v \neq w$ son tales que $m^t_{v,w} = 0$, en (3.23) queda la desigualdad que acabamos de probar que se cumple. En cambio, si $m^t_{v,w} = 1$ por (3.23) debe valer $p^t_w > p^t_v$. Si el camino activado es $w_0, w_1, \ldots, w_k$ con $w_0 = v_1, w_k = v_2$, este camino tiene k arcos, así que $\sum_{(v,w)\in E_{grid}} m^t_{v,w} = k$. Por lo tanto $p_{w_j} = j, \forall 0 \leq j \leq k$ es una asignación factible. □

Queremos ver que los puntos de inicio y fin de las etapas estén en la trayectoria de la base. Pasamos a probarlo.

Proposición 3.2.4. *Dada una solución factible para la grilla damero, si $v \in \mathcal{K}_{grid}$ satisface $L^t_v = 1$ o $R^t_v = 1$ para alguna etapa $t \in \tau$, entonces $v = orig$ o $\sum_{k\in\tau} \sum_{w\in V^{grid}_v} m^k_{w,v} \geq 1$.*

Demostración. Supongamos que $v \neq orig$ y veamos que se cumple la otra afirmación, a la que llamaremos *afirmación de llegada*. Lo hacemos por inducción sobre $t \in \tau$, suponiendo que $R^t_v = 1$. Para el caso base, con $k = 1 \in \tau$, tenemos que $v \neq orig$ y que $L^1_{orig} = 1$ por (3.3), así que $L^t_v = 0$ por (3.1). Así se sigue la afirmación de llegada por (3.7).
Ahora hacemos el paso inductivo. Suponiendo que $v \neq orig$ y que si un nodo v cumple $R^k_v = 1$ verifica la afirmación de llegada, veamos que también la satisface si cumple $R^{k+1}_v = 1$. Este nodo puede activar o no el inicio de la etapa $k + 1$. En caso afirmativo, se tiene $L^{k+1}_v = 1 = R^k_v$ por (3.5). Por hipótesis inductiva, v satisface la afirmación de llegada. Si $L^{k+1}_v = 0$, nuevamente por (3.7) v cumple la afirmación de llegada.
Ya completamos el caso donde $R^t_v = 1$. Falta el de $L^t_v = 1$. Si $t = 1 \in \tau$, por (3.3) sabemos que $v = orig$. Si $t > 1$, por (3.5) se sigue que $L^k_v = R^{k-1}_v = 1$, y v satisface la afirmación de llegada porque ya cubrimos este caso. □

Si usamos la grilla damero, la trayectoria de la base en una etapa tendrá una o más aristas y tenemos que garantizar de alguna manera que sea conexa. Estudiamos varias posibilidades para esto. Comenzamos con las SEC que se agregan en caso necesario, para romper subtours cíclicos que una solución haya generado. En este caso, no tenemos el mismo problema visto antes con la posibilidad de perder caminos válidos con un ciclo.

Como se busca minimizar la trayectoria de la base, siempre se podrá reemplazar un camino válido con un ciclo por uno sin ese ciclo.

$$\forall t \in \tau : \sum_{\{v,w\}:v,w\in S} m^t_{v,w} \leq |S| - 1, \forall S \subset \mathcal{K}$$

Como dijimos en la descripción del problema, se le prohíbe al vehículo base entrar o salir de los grafos y circular por ellos. Por eso sólo existen variables m para las aristas de la grilla.

Damos a continuación SEC alternativas, que indican que se debe ingresar desde afuera a un subtour que no pase por el inicio y salir fuera de uno que no pase por el retorno.

$$\forall t \in \tau : \sum_{v\notin S, w\in S} m^t_{v,w} \geq 1 - \sum_{v\in S} L^t_v, \forall S \subset \mathcal{K}_{grid}$$

$$\forall t \in \tau : \sum_{v\notin S, w\in S} m^t_{w,v} \geq 1 - \sum_{v\in S} R^t_v, \forall S \subset \mathcal{K}_{grid}$$

Ahora pasamos a las restricciones de grilla completa. Este caso no nos da problemas de conectividad porque se usará un solo arco por etapa (o bien no se usa ninguno y la base queda en el mismo lugar). El arco usado será el que va del nodo de inicio al nodo de retorno. Una posibilidad para implementar esta regla es utilizar las siguientes dos ecuaciones. En caso de agregarlas, podríamos prescindir de (3.7) para el modelo de grilla completa.

$$\forall v, w \in \mathcal{K}_{grid}/v \neq w, t \in \tau : m^t_{v,w} \geq L^t_v + R^t_w - \frac{3}{2} \qquad (1)$$

$$\forall v, w \in \mathcal{K}_{grid}/v \neq w, t \in \tau : m^t_{v,w} \leq \frac{1}{2}(L^t_v + R^t_w) \qquad (2)$$

Queremos que solamente se use el arco del inicio al retorno si son puntos diferentes, y ninguno si son el mismo. Probaremos que esto ya está implicado.

Proposición 3.2.5. *Dada una solución factible del modelo con grilla completa, para una etapa $t \in \tau$ sean v_1, v_2 tales que $L^t_{v_1} = 1, R^t_{v_2} = 1$. Entonces, dado $(v, w) \in E_{grid}$, vale que*

$$m^t_{v,w} = 1 \Leftrightarrow v = v_1 \wedge w = v_2$$

Demostración. Por (1) sabemos que $m^t_{v_1,v_2} = 1$ si $v_1 \neq v_2$. Por otro lado, si $v \neq v_1$ resulta $L^t_v = 0$ por (3.1), y si $w \neq v_2$ sabemos que $R^t_w = 0$ por (3.2). Por lo tanto, para todo arco que no vaya de v_1 a v_2 se sigue que $m^t_{v,w} = 0$ por (2). □

Aunque estas restricciones imponen que se utilice a lo sumo un arco por etapa y sea el correcto sin necesidad de utilizar (3.7), hemos comprobado en la práctica que incrementan enormemente los tiempos de ejecución del modelo, como detallaremos en la sección de experimentación.

Por eso, utilizamos una alternativa computacionalmente más eficiente, y probaremos que cumple el mismo objetivo. Consiste en reincorporar (3.7) y agregar una imposición explícita de utilizar a lo sumo un arco por etapa:

$$\forall t \in \tau : \sum_{v,w\in\mathcal{K}_{grid}, v\neq w} m^t_{v,w} \leq 1 \qquad (3.24)$$

Proposición 3.2.6. *Las restricciones* (3.7) *y* (3.24) *garantizan que, dados* $t \in \tau$ *y* $v, w \in \mathcal{K}_{grid}, v \neq w$*, se verifique*

$$m^t_{v,w} = 1 \Leftrightarrow L^t_v = 1 \wedge R^t_w = 1$$

Demostración. Dada $t \in \tau$, sean $v, w \in \mathcal{K}_{grid}$ tales que $L^t_v = R^t_w = 1$.
Si se tiene $v = w$, debemos ver que $m^t_{q,s} = 0, \forall q, s \in \mathcal{K}_{grid}, q \neq s$. En efecto, en (3.7) se observa que $L^t_q - R^t_q = 0, \forall q \in \mathcal{K}_{grid}$, puesto que si $q = v$ se restan unos, y si $q \neq v$ se restan ceros. Para $q \in \mathcal{K}_{grid}$, si existiera $s \in \mathcal{K}_{grid}, s \neq q$ tal que $m^t_{q,s} = 1$, para que la sumatoria fuera 0 se necesitaría al menos un $r \in \mathcal{K}_{grid}, r \neq q$ tal que $m^t_{r,q} = 1$. Pero como $(r, q), (q, s)$ son aristas diferentes, tendríamos al menos dos variables activadas para la etapa t, y esto contradice (3.24).
Falta ver el caso $v \neq w$. Tenemos $L^t_v - R^t_v = 1, L^t_w - R^t_w = -1$, y esta suma da 0 para cualquier otro nodo. En el caso de v, para cumplir (3.7) se requiere al menos un $q \in \mathcal{K}_{grid}, q \neq v$ tal que $m^t_{v,q} = 1$ (y es exactamente uno por (3.24)). Por la misma razón se verifica $m^t_{s,v} = 0, \forall s \in \mathcal{K}_{grid}, s \neq v$. Análogamente, existe un único $s \in \mathcal{K}_{grid}, s \neq w$ tal que $m^t_{s,w} = 1$ y se aprecia que $m_{w,s} = 0, \forall s \in \mathcal{K}_{grid}, s \neq w$. Como se verifica que $m_{v,q} = m_{s,w} = 1$, por (3.24) resulta que $(v, q) = (s, w)$; en resumen, el único arco activado que pasa por v o w es el que los conecta. Para verificar que ningún otro arco se activa, para $q \in \mathcal{K}_{grid}, q \neq v, q \neq w$ la ecuación (3.7) requiere que no se active ninguna arista que pase por q o que se activen al menos dos (razonamiento análogo al caso $v = w$). Luego, no se activa ninguna que pase por q. □

3.2.4. Reducción de soluciones redundantes

Queremos eliminar del sistema soluciones equivalentes que no aportan mejoras a la función objetivo, para que el espacio de soluciones no sea más grande de lo necesario. Primero, fijamos un orden de salida de los drones para cada etapa. Para que salga uno, tienen que haber salido los anteriores.

$$\forall t \in \tau_P, 1 \leq d \leq |\mathcal{D}| - 1 : \sum_{g \in \mathcal{G}} \mu^{d,t}_g \geq \sum_{g \in \mathcal{G}} \mu^{d+1,t}_g \tag{3.25}$$

Definimos la cantidad de grafos visitados durante la etapa t:

$$\forall t \in \tau : k_t = \sum_{d \in \mathcal{D}} \sum_{g \in \mathcal{G}} \mu^{d,t}_g \tag{3.26}$$

Sean $\beta_t, t \in \tau$ variables binarias que indican si, al empezar una etapa, todos los grafos ya fueron explorados.

$$\beta_1 = 0 \tag{3.27}$$

$$\forall 1 < t \leq |\tau| : \beta_t \leq \frac{1}{|\mathcal{G}|} \sum_{j=1}^{t-1} k_j \tag{3.28}$$

$$\forall 1 < t \leq |\tau| : \beta_t \geq 1 - |\mathcal{G}| + \sum_{j=1}^{t-1} k_j \tag{3.29}$$

Ahora podemos concentrar la exploración en las primeras etapas y prohibir que en una etapa no salga ningún dron si falta explorar grafos.

$$\forall t \in \tau_P : \sum_{d \in \mathcal{D}} \sum_{g \in \mathcal{G}} \mu^{d,t}_g \geq 1 - \beta_t \tag{3.30}$$

3.2.5. Función objetivo

Finalmente, podemos establecer el problema de optimización, distinguiendo si usamos grilla damero o grilla completa. Recordamos que el objetivo era minimizar la duración total de la operación de monitoreo. Las restricciones para las cuales no se especifica el tipo de grilla se usan en todos los casos.

$$\begin{aligned} \text{mín} \ & \sum_{t \in \tau} h_t \\ \text{sujeto a} \ & (3.1) - (3.20) \\ & (3.21) - (3.23) \text{ si es grilla damero} \\ & (3.24) \text{ si es grilla completa} \\ & (3.25) - (3.30) \end{aligned}$$

En la sección de experimentación vamos a comparar el desempeño en la práctica de la grilla completa 1 y la grilla damero. Mostraremos que las dos alternativas tienen los mismos valores factibles.

Proposición 3.2.7. *Dados un conjunto de grafos, una grilla y una cantidad de drones disponibles, los modelos de grilla damero y grilla completa 1 tienen el mismo valor óptimo en la función objetivo.*

Demostración. Para probar esto, dada una solución factible de la grilla damero construimos una solución factible de la grilla completa 1 con el mismo valor objetivo y viceversa. Dada una etapa $t \in \tau$, consideramos los nodos $v, w \in \mathcal{K}_{grid}$ tales que $L_v^t = R_w^t = 1$. Si $v = w$, vimos que ninguna arista $m_{q,p}^t$ de la grilla damero se activa. Luego, podemos construir una solución factible de la grilla completa donde $m_{q,p}^t = 0, \forall q, p \in \mathcal{K}_{grid}$, dejando todas las demás variables con los mismos valores (y eliminando las variables p de orden de recorrido). La h_t de esta etapa que se obtenía de la solución factible en grilla damero también funciona para la solución factible de grilla completa, ya que en ambos casos la base recorre distancia 0 y las trayectorias de los drones se mantienen idénticas. Como este razonamiento vale para todas las etapas, el valor de la función objetivo no se modifica. Si $v \neq w$, podemos suponer que la longitud total de las aristas $m_{q,p}^t$ activadas es la distancia 1 de v a w. En efecto, no puede ser menor, y si fuera mayor se podría reemplazar este camino por uno igual a la distancia 1, manteniendo igual o mejorando h_t. Luego, convertimos a esta solución factible de grilla damero en una de grilla completa 1 donde $m_{v,w} = 1$. Como el costo de esta arista es la distancia 1, la variable h_t se mantiene igual.

Lo mismo vale a la inversa. En una solución factible de la grilla completa 1, el camino de la base en cada etapa entre los nodos v, w de inicio y retorno (la arista activada $m_{v,w}^t$) se cambia por un camino de aristas activadas que suma la distancia 1, y nuevamente h_t se mantiene igual. □

A continuación probamos otra propiedad que sirve a la comparación de resultados con los diferentes modelos: que la grilla completa 2 da óptimos menores o iguales a la grilla completa 1 y la grilla damero.

Proposición 3.2.8. *Dada una instancia, El valor óptimo de la función objetivo que se obtiene con la grilla completa 2 es menor o igual al que se obtiene con la grilla completa 1 y la grilla damero.*

Demostración. Para verificarlo, mostraremos que dado un valor h_t factible para grilla completa 1 o grilla damero, se puede igualar o mejorar con grilla completa 2. Dados los nodos $v, w \in \mathcal{K}_{grid}$ de inicio y de retorno asociados para el valor h_t, armamos una solución factible para grilla completa 2 tomando las mismas trayectorias de los drones y con los mismos inicio y retorno v, w pero con distancia 2 entre ellos. En efecto esta nueva configuración tiene al h_t original como valor factible porque el tiempo de viaje de los drones no cambia (para ellos las distancias se miden igual en cualquier modelo); y el tiempo de viaje de la base con distancia 2 entre los mismos nodos es igual o más corto, ya que la distancia euclídea es la menor posible. Por lo tanto, el mismo h_t original que era cota superior de los tiempos de viaje de la base y de los drones con la distancia 1 funciona también como cota superior para la base con distancia 2. Por lo tanto, el mismo h_t resulta factible con grilla completa 2. En particular, el h_t óptimo con este último modelo es no mayor al de los otros. □

En la sección de experimentación vamos a utilizar diferentes discretizaciones, es decir conjuntos de nodos de la grilla. Dadas dos discretizaciones 1,2 de una misma instancia con conjuntos $\mathcal{K}^1_{grid}, \mathcal{K}^2_{grid}$, diremos que la discretización 2 es un *refinamiento* de la 1 si las dos utilizan el mismo modelo entre los tres estudiados, se verifica $\mathcal{K}^1_{grid} \subset \mathcal{K}^2_{grid}$ y además todos los arcos de la primera grilla están en la segunda, manteniéndose completos o subdivididos con nodos intermedios. Probaremos que una discretización ofrece cotas superiores para los valores de la función objetivo de un refinamiento.

Proposición 3.2.9. *Dadas una instancia y una etapa $t \in \tau$, sean dos discretizaciones 1,2 tales que la discretización 2 es un refinamiento de la 1. Entonces, todo valor h_t factible para la discretización 1 es factible para la discretización 2.*

Demostración. Si el modelo es de grilla completa, un h_t factible de la discretización 1 corresponde a la distancia entre el nodo de inicio y el de retorno (ambos en $\mathcal{K}^1_{grid}$). Como los dos nodos están en $\mathcal{K}^2_{grid}$, h_t también es factible para la discretización 2, puesto que la base puede realizar el mismo camino. Si el modelo es de grilla damero, dado el camino del inicio al retorno que corresponde al valor h_t en la discretización 1, en la 2 se puede hacer el mismo camino, tal vez pasando por nuevos nodos intermedios (puesto que los mismos arcos debían mantenerse completos o estar subdivididos). Por lo tanto, h_t también es factible para la discretización 2. □

De esto se sigue que la solución óptima de la discretización 2 es menor o igual a la de la discretización 1.

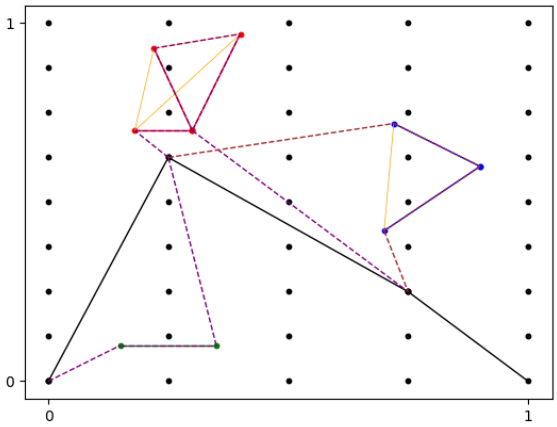
4. EXPERIMENTACIÓN

Ahora ponemos a prueba las tres variantes con varias instancias, cuya construcción detallaremos. La grilla completa 1 y la grilla damero serán comparadas para estudiar qué ocurre con los tiempos de ejecución cuando se tiene más variables o más restricciones. La grilla completa 2 está para estudiar casos donde el vehículo base puede moverse libremente por el plano. Mostramos a continuación los caminos óptimos encontrados por Gurobi para el ejemplo que utilizamos para graficar la construcción del modelo. Los arcos de la grilla fueron eliminados para reducir ruido. En la Figura 4.1, las líneas negras representan el camino del vehículo base. Se disponía de dos drones, cuyos caminos están representados por líneas punteadas. Uno está marcado en marrón y el otro en violeta. Observamos que en los óptimos de grilla completa 1 y grilla damero el vehículo base recorre la misma distancia total.

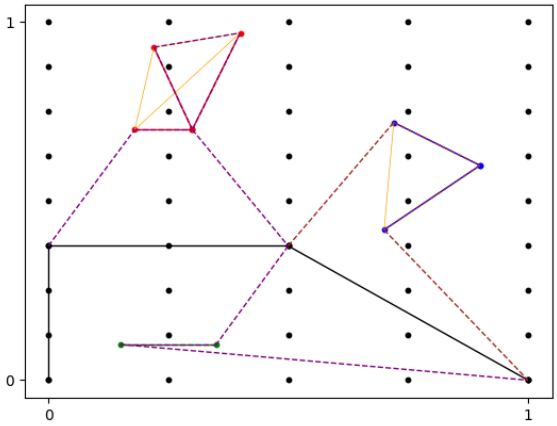
4.1. Construcción de las instancias

Comenzamos contando cómo se construyen las instancias que se arman en el paper original para evaluar el desempeño de sus modelos, ya que nosotros utilizamos las mismas para esta finalidad. Contamos con ellas puesto que encontramos el código que las genera en un repositorio GitHub público de Carlos Valverde, coautor del trabajo mencionado. Aseguramos que son las mismas porque el código tiene *random seed* implementado.

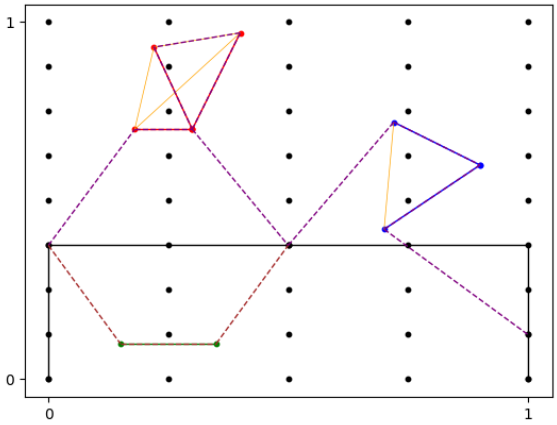
Se toma un sector cuadrado del plano con 100 unidades de lado, este se subdivide en cuadrados pequeños y dentro de algunos de ellos, seleccionados al azar, se construyen grafos con forma de grilla, cuyos nodos también tienen ubicación aleatoria y ligeramente perturbada. Se da más detalles en la Sección 6 de [Amorosi et al., 2023]. Como ejemplo se puede ver la Figura 13 en esa misma sección. Mediante este método se construye 5 instancias con 5 grafos y 5 instancias con 10 grafos. Las cantidades de nodos con que se arman estos grafos son 4, 6, 8, 10 y 12, igualmente distribuidas entre los grafos que hay en cada instancia. Así quedan 10 instancias en total. Sobre cada una se utilizan todas las combinaciones posibles de una cantidad de drones dentro del conjunto $\{1, 2, 3\}$ y un valor $N_{\mathcal{D}}$ de autonomía del dron en el conjunto $\{20, 30, 40, 50, 60\}$. Sobre cada una de estas posibles combinaciones, se promedia los resultados de los 5 mapas con 5 grafos y por otro lado se promedia los resultados de los 5 mapas con 10 grafos. En los 10 mapas se tiene $orig = dest = (50, 50)$. Además en el paper se define $v_{\mathcal{D}} = 2v_M$. Nosotros usaremos este criterio. No obstante, notamos que en el código encontrado los valores por defecto eran $v_M = 1, v_{\mathcal{D}} = 3$. En nuestro caso elegimos $v_M = 1{,}5, v_{\mathcal{D}} = 3$. Además se prueba en cada combinación posible el resultado según si hay que cubrir un porcentaje de cada arista o un porcentaje de la suma de longitudes de aristas de cada grafo, pero esto es ajeno a lo que hacemos en nuestro trabajo. Los autores del paper de referencia implementaron los modelos en el solver comercial Gurobi, y en esta tesis hacemos lo mismo. Además ejecutamos nuestro modelo de programación lineal entera mixta con un tiempo límite de media hora para cada instancia a diferencia del trabajo mencionado, donde fue de una hora. Esta media hora fue el tiempo total que se dio para la construcción del modelo MILP y su resolución.



(a) Grilla completa 2



(b) Grilla completa 1



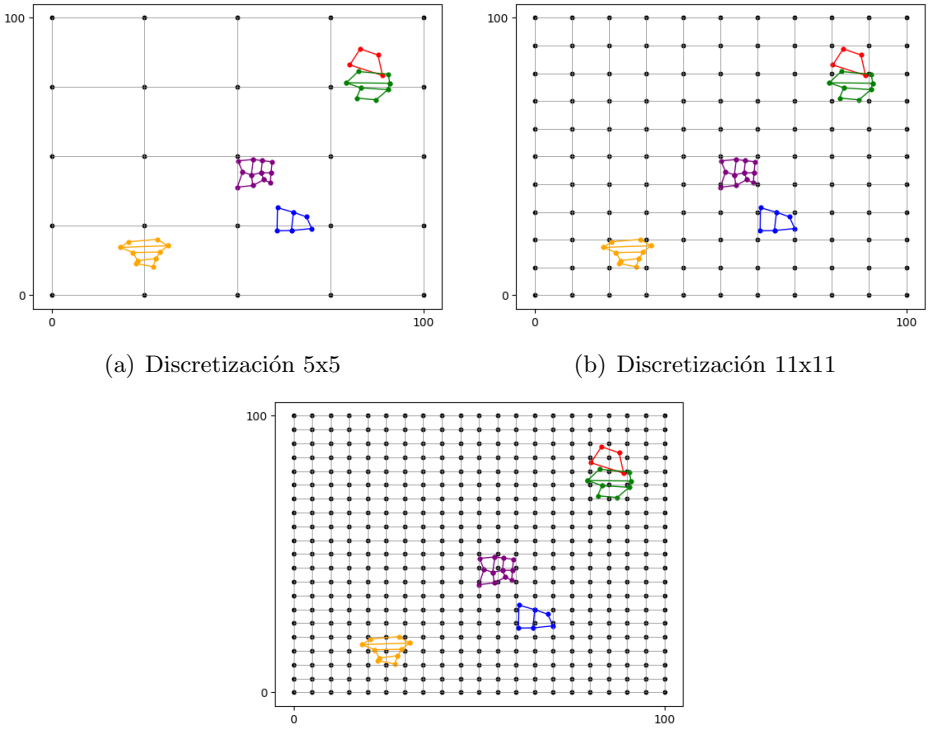
(c) Grilla damero

Fig. 4.1: Soluciones óptimas del ejemplo con los tres modelos.

Recordamos que nuestro modelo fue implementado con 3 formatos diferentes: grilla damero, grilla completa 1 y grilla completa 2. Este último modelo es el más similar a los

que utilizaron los autores del paper, dado que ellos trabajaron asumiendo que el vehículo base podía moverse libremente por el plano y por lo tanto viajaba en línea recta entre el inicio y el fin de cada etapa; de nuestras opciones, la única que le da esta posibilidad es la grilla completa 2. Por otro lado, la grilla damero y la completa 1 son dos representaciones diferentes de una situación donde la base sólo puede viajar por las 'calles' de la grilla, y se diferencian en la cantidad de variables y de restricciones.

Evaluamos 6 instancias tomadas al azar (3 de 5 grafos y 3 de 10 grafos), con cada uno de los 3 modelos, con $N_{\mathcal{D}} = 60$, $|\mathcal{D}| = 1, 2, 3$ y con 3 discretizaciones diferentes. Estas discretizaciones son subdivisiones del cuadrado [0,100]×[0,100] que contiene los grafos con distancia entre filas y entre columnas de 25, de 10 y de 5. Estas son, respectivamente, discretizaciones 5x5, 11x11 y 21x21, puesto que los bordes del mapa son tomados como filas y columnas. Graficamos un ejemplo en la Figura 4.2. Esto nos da un total de 162 ejecuciones, que representan un máximo de 81 horas de cómputo. Algunas preguntas que



(a) Discretización 5x5

(b) Discretización 11x11

(c) Discretización 21x21

Fig. 4.2: Las tres discretizaciones para una instancia de experimentación.

buscaremos responder son:

- El grado de eficacia del modelo para diferentes discretizaciones.
- Entre la grilla damero y la completa 1, representaciones diferentes de una misma situación, cuál es más eficiente.
- Cómo influyen los diferentes parámetros en los tiempos de construcción y resolución.
- Qué alternativas se podría estudiar para mejorar el desempeño.

4.2. Resultados y análisis

Todos los modelos se construyeron y resolvieron utilizando Gurobi Optimizer version 11.0.3 build v11.0.3rc0 (win64 - Windows 10.0 (19045.2)), con modelo CPU: Intel(R) Core(TM) M-5Y10c CPU @ 0.80GHz y una memoria RAM de 4GB.

Presentaremos tres tablas de resultados, cada una para una discretización, y las analizaremos. Comenzamos describiendo sus atributos.

- ind: va de 0 a 4, es el índice entero que identifica la instancia junto con su cantidad de grafos (hay 5 con cada cantidad). Los índices tomados al azar son 0, 2, 4.
- G: cantidad de grafos. Identifica la instancia junto con ind.
- D: cantidad de drones disponibles.
- mod: el modelo utilizado en cada instancia. Se tiene dam para el damero y c1, c2 para la grilla completa 1 y 2 respectivamente.
- estado: la forma de terminación de la ejecución en Gurobi. Las dos posibilidades son óptimo (resuelto a optimalidad dentro del tiempo límite) y agotado (la ejecución llegó al tiempo límite).
- cota_inf: una cota inferior del valor óptimo que da Gurobi. Si da 0, no se encontró una mejor que la trivial (los valores de tiempo siempre son no negativos).
- cota_sup: cota superior de la solución óptima encontrada por Gurobi (igual al óptimo si se encontró); está dada por una solución factible. Si no se encontró ninguna dentro del tiempo límite, se registra inf.
- gap: es un valor real entre 0 y 1. Es la diferencia proporcional que calcula Gurobi entre la cota superior y la inferior. Se registra inf si no se encontró una solución factible.
- fact: cantidad de soluciones factibles que Gurobi encontró dentro del tiempo límite.
- t_const(s): el tiempo que tardó la construcción del modelo, expresado en segundos.
- t_ejec(s): el tiempo de ejecución de la resolución del modelo, expresado en segundos. Entre este valor y el anterior suman como máximo alrededor de 1800s (30 minutos).

ind	**G**	**D**	**mod**	**estado**	**cota_inf**	**cota_sup**	**gap**	**fact**	**t_const(s)**	**t_ejec(s)**
0	5	1	dam	óptimo	153.61	153.61	0	10	5.01	35.23
4	5	1	dam	óptimo	148.89	148.89	0	10	6.42	41.28
2	5	1	dam	óptimo	189.17	189.17	0	10	6.56	72.39
0	10	1	dam	agotado	263.95	285.55	0.08	10	17.7	1782.74
4	10	1	dam	agotado	264.9	325.66	0.19	10	26.24	1773.98
2	10	1	dam	agotado	257.59	305.19	0.16	10	19	1781.7
0	5	1	c1	óptimo	153.61	153.61	0	10	4.41	48.99
4	5	1	c1	óptimo	148.89	148.89	0	9	4.22	56.67

2	5	1	c1	óptimo	189.17	189.17	0	10	4.22	99.83
0	10	1	c1	agotado	259.34	280.98	0.08	10	10.49	1789.64
4	10	1	c1	agotado	269.07	335.06	0.2	9	11.46	1788.8
2	10	1	c1	agotado	257.77	313.42	0.18	10	13.38	1786.72
0	5	1	c2	óptimo	149.01	149.01	0	10	3.83	39.68
4	5	1	c2	óptimo	141.02	141.02	0	8	3.5	29.56
2	5	1	c2	óptimo	187.25	187.25	0	8	3.45	55.95
0	10	1	c2	agotado	260.03	275.41	0.06	10	10.74	1789.4
4	10	1	c2	agotado	272.96	inf	inf	0	12.55	1787.76
2	10	1	c2	agotado	259.73	297	0.13	10	12.76	1787.37
0	5	2	dam	óptimo	110.32	110.32	0	10	12.57	1041.82
4	5	2	dam	óptimo	98.97	98.97	0	7	8.02	472.62
2	5	2	dam	óptimo	135.68	135.68	0	10	5.56	246.66
0	10	2	dam	agotado	121.27	235.59	0.49	10	18.11	1782.01
4	10	2	dam	agotado	125.16	232.48	0.46	10	18.64	1781.46
2	10	2	dam	agotado	120.42	232.06	0.48	10	18.58	1781.54
0	5	2	c1	óptimo	110.32	110.32	0	10	3.89	480.33
4	5	2	c1	óptimo	98.97	98.97	0	10	3.97	228.33
2	5	2	c1	óptimo	135.68	135.68	0	8	3.83	290.53
0	10	2	c1	agotado	116.33	210.43	0.45	2	61.68	1738.7
4	10	2	c1	agotado	124.02	inf	inf	0	20.72	1779.43
2	10	2	c1	agotado	119.55	inf	inf	0	30.62	1769.53
0	5	2	c2	óptimo	109.25	109.25	0	5	4.96	333.36
4	5	2	c2	óptimo	95.09	95.09	0	6	4.03	342.72
2	5	2	c2	óptimo	135.68	135.68	0	10	5.7	341.59
0	10	2	c2	agotado	116.19	191.16	0.39	10	15.92	1784.39
4	10	2	c2	agotado	123.9	252.62	0.51	5	23.53	1776.73
2	10	2	c2	agotado	118.54	inf	inf	0	24.07	1776.01
0	5	3	dam	óptimo	77.69	77.69	0	10	8.07	1597.7
4	5	3	dam	agotado	66.67	72.73	0.08	10	18.87	1781.23
2	5	3	dam	óptimo	108.89	108.89	0	10	7.83	1134.82
0	10	3	dam	agotado	78.23	187.24	0.58	6	32.76	1767.37
4	10	3	dam	agotado	82.34	193.65	0.57	10	26.23	1773.84
2	10	3	dam	agotado	77.05	245.85	0.69	7	27.46	1772.87
0	5	3	c1	óptimo	77.69	77.69	0	10	8.09	1461.5
4	5	3	c1	óptimo	72.73	72.73	0	10	6.17	732.39
2	5	3	c1	agotado	72.95	108.89	0.33	10	6.26	1793.86
0	10	3	c1	agotado	78.59	221.37	0.64	3	23.47	1776.78
4	10	3	c1	agotado	82.78	inf	inf	0	29	1771.07
2	10	3	c1	agotado	78.24	189.66	0.59	7	24.56	1775.56
0	5	3	c2	óptimo	77.69	77.69	0	5	7.01	809.07
4	5	3	c2	óptimo	72.73	72.73	0	10	5.98	920.8
2	5	3	c2	agotado	64.82	108.89	0.4	10	6.47	1793.62
0	10	3	c2	agotado	77.32	169.21	0.54	7	24.1	1776.15
4	10	3	c2	agotado	79.45	253.32	0.69	8	22.79	1777.4

2	10	3	c2	agotado	76.45	195.3	0.61	3	29.88	1770.46

Tab. 4.1: Resultados para la discretización 5x5.

Comenzamos examinando los resultados de la Tabla 4.1. Primero notamos una correlación casi absoluta entre el estado y la cantidad de grafos. En general, las instancias de 5 grafos se resuelven a optimalidad y las de 10 grafos agotan el tiempo. Esto es aún más notorio en las instancias con un dron, donde vemos que las de 5 grafos se resuelven en menos de 3 minutos y las de 10 grafos agotan los 30. Esto sugiere que la cantidad de grafos es el factor de mayor peso en el tiempo de resolución. Esta deducción es coherente con el hecho de que la cantidad de grafos también determina la de etapas (es el doble de la cantidad de grafos). Por otro lado, casi todas las instancias con un solo dron disponible o con 5 grafos presentan gaps menores o iguales a 0.2. Es posible que a aquellas no resueltas dentro del tiempo límite no les faltara mucho tiempo. Como regla general, las instancias con 10 grafos presentan mayores dificultades, con gaps que suelen ubicarse entre 0.3 y 0.6, y en algunos casos no aparece una solución factible dentro del tiempo límite. Más adelante realizaremos un análisis comparativo de los tiempos de construcción y de ejecución para comparar el desempeño de los modelos.

En la Tabla 4.1 también se puede verificar que en todas las instancias donde se encontró óptimo para la grilla completa 1 y la grilla damero, este valor es el mismo. Además, en aquellos casos donde también se ve un óptimo para la grilla completa 2, este es menor o igual al del damero y la grilla completa 1. Esto ya había sido anticipado en las Proposiciones 3.2.7 y 3.2.8.

ind	**G**	**D**	**mod**	**estado**	**cota_inf**	**cota_sup**	**gap**	**fact**	**t_const(s)**	**t_ejec(s)**
0	5	1	dam	óptimo	148.23	148.23	0	10	25.51	911.88
4	5	1	dam	óptimo	140.23	140.23	0	10	31.79	588.93
2	5	1	dam	óptimo	181.94	181.94	0	10	33.79	1299.9
0	10	1	dam	agotado	227.5	269.45	0.16	10	70.21	1729.96
4	10	1	dam	agotado	240.24	inf	inf	0	57.86	1742.43
2	10	1	dam	agotado	228.03	inf	inf	0	54.17	1745.93
0	5	1	c1	agotado	142.03	148.38	0.04	10	14.19	1786.27
4	5	1	c1	agotado	129.14	140.23	0.08	10	22.7	1777.7
2	5	1	c1	agotado	145.96	181.94	0.2	10	18.9	1781.34
0	10	1	c1	agotado	215.37	296.97	0.27	10	47.95	1753.12
4	10	1	c1	agotado	217.55	inf	inf	0	43.16	1757.25
2	10	1	c1	agotado	207.56	378.78	0.45	5	69.03	1731.53
0	5	1	c2	agotado	140.66	146.12	0.04	10	14.98	1785.45
4	5	1	c2	agotado	132.24	137.57	0.04	10	21.11	1779.46
2	5	1	c2	agotado	168.4	179.06	0.06	10	23.61	1776.64
0	10	1	c2	agotado	213.09	inf	inf	0	53.59	1746.81
4	10	1	c2	agotado	217.26	inf	inf	0	61.9	1738.43
2	10	1	c2	agotado	208.99	inf	inf	0	52.36	1747.97
0	5	2	dam	agotado	65.43	111.07	0.41	6	39.89	1760.41
4	5	2	dam	óptimo	96.39	96.39	0	9	34.99	1412.03

2	5	2	dam	agotado	103.64	131.16	0.21	10	34.84	1765.34
0	10	2	dam	agotado	103.88	296.38	0.65	3	107.07	1693.34
4	10	2	dam	agotado	106.24	291.5	0.64	2	105.76	1694.53
2	10	2	dam	agotado	100.49	215.61	0.53	3	109.91	1690.44
0	5	2	c1	agotado	70.37	107.7	0.35	10	26.66	1773.6
4	5	2	c1	agotado	67.5	96.39	0.3	9	27.16	1773.24
2	5	2	c1	agotado	69.52	140.48	0.51	9	26.73	1773.53
0	10	2	c1	agotado	103.78	inf	inf	0	89.36	1710.75
4	10	2	c1	agotado	104.8	inf	inf	0	90.06	1710.14
2	10	2	c1	agotado	100.69	inf	inf	0	109.77	1690.34
0	5	2	c2	agotado	67.91	107.39	0.37	5	27.1	1773.28
4	5	2	c2	agotado	69.94	95.59	0.27	10	27.65	1772.76
2	5	2	c2	agotado	66.45	134.6	0.51	4	26.96	1773.46
0	10	2	c2	agotado	103.48	inf	inf	0	94.56	1705.59
4	10	2	c2	agotado	104.33	inf	inf	0	92.98	1707.19
2	10	2	c2	agotado	100.38	inf	inf	0	98.69	1701.48
0	5	3	dam	agotado	40.81	77.69	0.47	4	44.12	1756.1
4	5	3	dam	agotado	42.46	78.02	0.46	6	46.87	1753.42
2	5	3	dam	agotado	49.94	inf	inf	0	46.63	1753.54
0	10	3	dam	agotado	69.5	inf	inf	0	146.65	1653.47
4	10	3	dam	agotado	70.97	inf	inf	0	114.57	1685.87
2	10	3	dam	agotado	67.86	inf	inf	0	112.97	1687.12
0	5	3	c1	agotado	37.61	83.38	0.55	9	28.94	1771.35
4	5	3	c1	agotado	42.24	82.96	0.49	8	31.66	1768.89
2	5	3	c1	agotado	47.61	146.68	0.68	1	39.34	1761.16
0	10	3	c1	agotado	69.5	inf	inf	0	142.05	1658.3
4	10	3	c1	agotado	70.26	inf	inf	0	126.99	1673.13
2	10	3	c1	agotado	67.36	inf	inf	0	127.71	1672.46
0	5	3	c2	agotado	38.53	74.51	0.48	7	36.13	1764.4
4	5	3	c2	agotado	40.79	81.42	0.5	10	35.41	1765
2	5	3	c2	agotado	42.9	inf	inf	0	35.8	1764.39
0	10	3	c2	agotado	69.27	inf	inf	0	126.01	1674.42
4	10	3	c2	agotado	70.12	inf	inf	0	125.79	1674.41
2	10	3	c2	agotado	67.45	inf	inf	0	127.71	1672.46

Tab. 4.2: Resultados para la discretización 11x11.

Desde esta discretización ya se puede ver que el modelo MILP afronta grandes dificultades para resolver este problema. Sólo se logra resolver a optimalidad dentro del tiempo límite las instancias con 5 grafos y 1 dron disponible, las cuales tienen el menor tamaño en cuanto a variables y restricciones. No obstante, vemos una situación intermedia entre la tabla anterior y la siguiente: en muchos casos se encuentra una solución factible, y para los casos con un solo dron se tiene gaps bajos (menores a 0.2).

ind	**G**	**D**	**mod**	**estado**	**cota_inf**	**cota_sup**	**gap**	**fact**	**t_const(s)**	**t_ejec(s)**

0	5	1	dam	agotado	111.39	148.86	0.25	10	100.35	1700.26
4	5	1	dam	agotado	113.82	159.07	0.28	9	100.74	1699.8
2	5	1	dam	agotado	124.19	inf	inf	0	99.94	1700.39
0	10	1	dam	agotado	211.03	inf	inf	0	273.26	1526.93
4	10	1	dam	agotado	216.68	inf	inf	0	274.67	1525.47
2	10	1	dam	agotado	206.92	inf	inf	0	273	1527.23
0	5	1	c1	agotado	95.69	inf	inf	0	169.25	1631.34
4	5	1	c1	agotado	107.19	inf	inf	0	161.51	1638.7
2	5	1	c1	agotado	115.65	inf	inf	0	133.29	1666.98
0	10	1	c1	agotado	210.79	inf	inf	0	336.81	1463.78
4	10	1	c1	agotado	215.27	inf	inf	0	357.35	1443.24
2	10	1	c1	agotado	205.13	inf	inf	0	345.83	1454.87
0	5	1	c2	agotado	97.31	inf	inf	0	138.85	1661.46
4	5	1	c2	agotado	109.2	inf	inf	0	133.05	1667.19
2	5	1	c2	agotado	113.93	inf	inf	0	133.41	1667.01
0	10	1	c2	agotado	0	inf	inf	0	354.26	1447.11
4	10	1	c2	agotado	215	inf	inf	0	418.47	1382.07
2	10	1	c2	agotado	205.11	inf	inf	0	339.75	1460.83
0	5	2	dam	agotado	52.67	143.94	0.63	4	141.64	1658.87
4	5	2	dam	agotado	53.87	100.99	0.47	6	157.58	1643.3
2	5	2	dam	agotado	63.48	138.81	0.54	5	158.68	1641.68
0	10	2	dam	agotado	101.26	inf	inf	0	466.39	1333.8
4	10	2	dam	agotado	0	inf	inf	0	472.49	1330.02
2	10	2	dam	agotado	0	inf	inf	0	440.25	1360.38
0	5	2	c1	agotado	47.33	inf	inf	0	161.94	1638.4
4	5	2	c1	agotado	52.24	158.26	0.67	2	162.39	1638.82
2	5	2	c1	agotado	56.23	inf	inf	0	166.12	1634.24
0	10	2	c1	agotado	0	inf	inf	0	559.27	1241.53
4	10	2	c1	agotado	0	inf	inf	0	499.28	1301.35
2	10	2	c1	agotado	0	inf	inf	0	554.68	1246.08
0	5	2	c2	agotado	0	inf	inf	0	213.4	1586.88
4	5	2	c2	agotado	50.29	inf	inf	0	199.57	1600.85
2	5	2	c2	agotado	51.02	inf	inf	0	141.49	1658.77
0	10	2	c2	agotado	0	inf	inf	0	587.16	1213.98
4	10	2	c2	agotado	0	inf	inf	0	506.59	1294.09
2	10	2	c2	agotado	0	inf	inf	0	476.26	1324.55
0	5	3	dam	agotado	30.63	104.35	0.71	4	214.27	1586.29
4	5	3	dam	agotado	34.14	77.61	0.56	2	205.13	1595.6
2	5	3	dam	agotado	37.83	inf	inf	0	217.52	1582.59
0	10	3	dam	agotado	0	inf	inf	0	666.59	1133.79
4	10	3	dam	agotado	0	inf	inf	0	672.97	1127.41
2	10	3	dam	agotado	0	inf	inf	0	638.71	1161.65
0	5	3	c1	agotado	0	inf	inf	0	239.14	1561.14
4	5	3	c1	agotado	33.19	308.91	0.89	1	195.42	1609.08
2	5	3	c1	agotado	34.89	inf	inf	0	196.25	1604.07

0	10	3	c1	agotado	0	inf	inf	0	680.51	1120.36
4	10	3	c1	agotado	0	inf	inf	0	599.76	1201.27
2	10	3	c1	agotado	0	inf	inf	0	709.56	1091.33
0	5	3	c2	agotado	0	inf	inf	0	209.79	1590.53
4	5	3	c2	agotado	33.17	270.25	0.88	2	212.24	1589.63
2	5	3	c2	agotado	0	inf	inf	0	237.33	1563.14
0	10	3	c2	agotado	0	inf	inf	0	739.57	1061.37
4	10	3	c2	agotado	0	inf	inf	0	729.96	1071.69
2	10	3	c2	agotado	0	inf	inf	0	712.81	1088.03

Tab. 4.3: Resultados para la discretización 21x21.

En la Tabla 4.3 vemos que ningún caso fue resuelto a optimalidad, y sólo en muy pocos se logró encontrar soluciones factibles dentro del tiempo límite de 30 minutos. Además, en esos pocos casos hay un gap alto, de por lo menos 0.25. Inferimos que al software le faltaba mucho tiempo para poder resolver las instancias con esta discretización. Incluso vemos que en muchos casos la cota inferior hallada es 0; es decir, no se encontró una mejor que la trivial.

Cabe destacar que en la discretización 21x21 no sólo lidiamos con problemas de tiempo de resolución, sino también de memoria. De las instancias listadas en la Tabla 4.3, desde la número 28 (instancia (0,10), con 2 drones disponibles y grilla completa 1) hasta la última fue necesario modificar parámetros del objeto model de Gurobi porque la configuración por defecto agotaba la memoria RAM antes de poder dar una solución inicial. Concretamente, dispusimos que una vez usados 0.5GB se comenzara a guardar nodos del árbol branch and bound en disco en lugar de en la memoria RAM (lo cual ralentiza la ejecución pero expande el límite de memoria). Además, se modificó el algoritmo por defecto que ejecuta Gurobi, el cual consiste en buscar un nodo inicial mediante un método Barrier (punto interior) y desde él ejecutar Dual Simplex; en cambio, lo configuramos para usar Dual Simplex desde el principio. Esto también ahorra memoria a cambio de perder velocidad.

Otro factor que ya hemos mencionado en este trabajo es el tiempo de construcción y ejecución. Para empezar, hemos comprobado que la forma de escribir las restricciones en Gurobipy influye en los tiempos de construcción del modelo. Lo vemos en las mediciones de tiempo para construir el modelo para el ejemplo cuya solución se ve en la Figura 4.1, usando grilla completa 2 y 2 drones. Tomando el modelo MILP original (con las dos restricciones que luego cambiaríamos por otras nuevas), el primer código planteado lo construyó en 109.26 segundos en promedio (sobre 5 repeticiones). El mismo promedio para una nueva escritura más eficiente en Gurobipy da 80.58 segundos, una reducción del 26.25 %. Sin embargo, la modificación de las restricciones pesa mucho más. Usando el código más eficiente con este cambio, el tiempo promedio de construcción del modelo MILP es de 2.94 segundos. En cuanto a la resolución, dos repeticiones para el código original tardaron 460 y 454 segundos respectivamente, contra 16 y 21 segundos del código nuevo con las restricciones modificadas. Realizando una comparación del mismo tipo para una instancia de las que usamos en la experimentación (instancia (0,10), grilla completa 2, 2 drones), en la Tabla 4.3 vemos que su construcción tardó 587.16 segundos (poco menos de 10 minutos),

mientras que un intento de construirlo con el código anterior fue interrumpido al cabo de una hora (3600 segundos) sin haberse completado. Por lo tanto, inferimos que el cambio de restricciones fue fundamental para poder construir todos los modelos MILP requeridos y experimentar con ellos con el tiempo límite asignado.

De los resultados de las tres tablas concluimos que esta forma de discretización no es efectiva para resolver el problema de coordinación de drones con vehículos base en la práctica. No obstante, en un trabajo posterior se podría probar otras variantes y mecanismos que detallamos más abajo.

Ahora nos disponemos a realizar un análisis comparativo de los tiempos de construcción de los modelos según los diferentes parámetros. Lo haremos tomando promedios sobre todas las instancias de experimentación que tienen cada parámetro con el valor dado. Esto es posible porque todas las construcciones se completaron dentro del tiempo límite a diferencia de las resoluciones, que en muchos casos no sabemos cuánto tiempo tomarían. En la

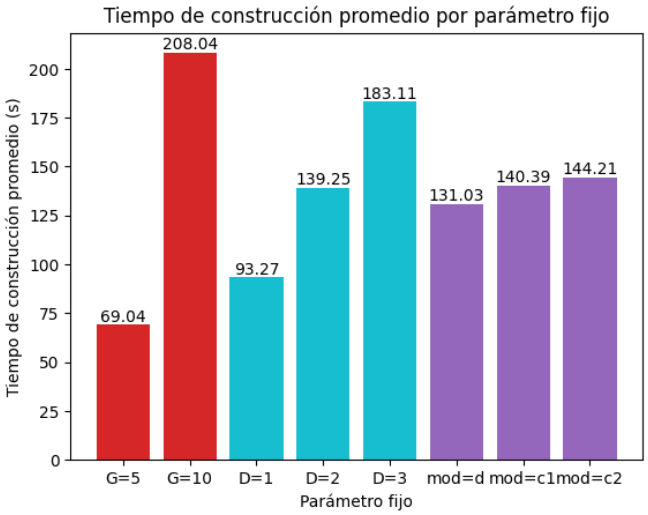


Fig. 4.3: Tiempo de construcción promedio en instancias de experimentación por parámetro.

Figura 4.3 podemos ver que con el doble de grafos el tiempo de construcción prácticamente se triplica, lo cual parece mostrar un crecimiento supralineal. El tiempo según la cantidad de drones disponibles crece de forma aproximadamente lineal. En cuanto al modelo, se ve promedios muy similares para la grilla completa 1 y la grilla completa 2, algo esperable ya que la única diferencia entre ambos modelos es la forma de calcular distancias en la grilla. El tiempo para la grilla damero resulta ligeramente menor al de los otros dos. Inferimos que hay un cierto grado de compensación entre las cantidades de variables y restricciones, que compararemos a continuación utilizando el ejemplo de la Figura 3.1. En primer lugar, vemos en la Tabla 4.4 que el modelo de grilla completa tiene muchas más variables que el de grilla damero. Esto se puede esperar porque el damero sólo agrega las variables de orden de recorrido por nodo y etapa, mientras que el de grilla completa conecta cada nodo de la grilla con todos los demás en cada etapa. Por otro lado, la grilla damero tiene más restricciones, pero la diferencia es mucho menor que la de cantidades de variables.

modelo	D	variables	restricciones
damero	1	4248	2879
damero	2	6756	3764
damero	3	9264	4649
completa 1	1	14946	1433
completa 1	2	17454	2318
completa 1	3	19962	3203

Tab. 4.4: Cantidades de variables y restricciones por modelo en el ejemplo graficado.

Como ya examinamos los tiempos de construcción promedio de todas las instancias, ahora nos enfocamos en los tiempos de resolución de los tres tipos de grilla. Los comparamos sólo en las instancias donde todas encontraron solución óptima dentro del tiempo límite. Lo hacemos de esta forma porque de lo contrario se introduciría un sesgo: un modelo que no pudo resolver una instancia dentro del tiempo límite cuando otro sí pudo no tendría ese tiempo extenso incluido en el cálculo, lo que beneficiaría a su promedio. Todas las instancias resueltas con los tres tipos de grilla tienen discretización 5x5 y 5 grafos. Dentro de estas, son todas las que usaban 1 o 2 drones y una de las que usaban 3 drones. Esta comparación dio un promedio de 701.86 segundos para la grilla damero, 380.88 segundos para la grilla completa 1 y 278.85 segundos para la completa 2. Por otro lado, hay 4 instancias de discretización 11x11 donde la grilla damero encontró solución óptima y las completas no. Vemos entonces que la grilla damero es la de resolución más lenta, seguida por la completa 1 y la más rápida es la completa 2. Comparando los dos primeros valores y viendo que los tiempos de construcción promedio por tipo de grilla de estas instancias son menores a 10 segundos, interpretamos que la representación de damero es menos eficiente computacionalmente que la de grilla completa 1 para resolver las instancias. Notamos una diferencia inesperada entre las dos versiones de grilla completa, que tienen las mismas variables y restricciones y sólo varían en el cálculo de distancias, aunque se las puede considerar similares si las comparamos con la grilla damero.

Para continuar el análisis con más casos, lo extendemos a todas las instancias donde estos dos modelos encontraron soluciones factibles, para comparar los gap generados y las cantidades de soluciones factibles halladas. En la Tabla 4.5 observamos que, cuando

modelo	casos con factibles	cant. factibles promedio	gap promedio
damero	37	7.92	0.29
completa 1	28	7.93	0.28
completa 2	25	7.72	0.26

Tab. 4.5: Soluciones factibles por modelo y sus resultados parciales.

se halla soluciones factibles, la cantidad de estas y el gap obtenidos dentro del tiempo límite son prácticamente iguales. Sin embargo, el modelo de grilla damero encuentra más soluciones factibles en ese tiempo. Concluimos que la grilla damero es más eficaz en la búsqueda de soluciones factibles que las completas, aunque estas últimas hallan soluciones óptimas más rápido.

Con estos resultados a la vista, cabe preguntarse cómo se podría mejorar el desempeño

de este método de discretización. En todo caso, no se debe esperar demasiado porque el problema en cuestión es difícil, y normalmente se afronta este tipo de situaciones construyendo heurísticas, por ejemplo algoritmos genéticos. En el paper de referencia se incluye una metaheurística además del programa convexo exacto que tomamos como base para nuestro trabajo. No obstante, quedan algunos caminos por intentar.

Por ejemplo, notamos que hemos planteado la coordinación de los drones con la base como un solo problema grande, aunque los dos tipos de vehículo tienen sus propias lógicas y caminos disponibles. La naturaleza de este problema nos permitiría precalcular una parte. Concretamente, se podría obtener el recorrido más corto de cada grafo, pasando por las aristas obligatorias, por cada posible par de nodos para comenzar y terminar este recorrido. Luego de tener todo esto precalculado, se le podría dejar al modelo general la tarea de encontrar los mejores caminos para la base y elegir los nodos de cada grafo por los cuales el dron entra y sale. Una vez hecha esta elección, se utilizaría el camino ya precalculado del dron por el grafo. Se podría plantear esta tarea previa como una variante del Rural Postman Problem. Como hay muchos problemas a resolver pero cada uno de ellos es pequeño, tal vez se pueda mejorar los tiempos de resolución globales.

Otra alternativa, iluminada por los resultados experimentales que obtuvimos, sería utilizar soluciones óptimas o buenas de discretizaciones malas como puntos de partida para discretizaciones más finas. Aunque la discretización 21x21 es un refinamiento de la 11x11 pero esta no lo es de 5x5, los buenos resultados obtenidos en esta última muestran que una mala discretización puede dar buenas soluciones rápido. Como ya vimos en la Proposición 3.2.9, una solución factible u óptima de una discretización puede traducirse a una solución factible de un refinamiento. Se puede también construir el código de forma que se facilite esta traducción. El concepto de darle a un modelo una solución factible para que comience a trabajar desde ella se conoce como *warm start*. Sería interesante comparar el desempeño (en tiempo y calidad de soluciones) de una discretización fina trabajando desde cero con el de un proceso de refinamientos y traducciones guiado por las soluciones factibles halladas en el camino.

5. CONCLUSIONES

En este trabajo hemos comenzado estudiando el estado actual de la investigación sobre problemas de optimización que involucran drones. Luego, nos concentramos en un trabajo concreto sobre coordinación entre UAVs y un vehículo base para una operación de monitoreo sobre diferentes zonas representadas como grafos. Tomamos como punto de partida un modelo exacto de programación convexa que se construyó para minimizar el tiempo total que tomaba completar la operación; y decidimos discretizar el problema y construirlo como un problema de optimización combinatoria, para el cual planteamos un modelo MILP.

Hemos construido tres versiones alternativas para la grilla, demostrado por qué el modelo resuelve el problema que formulamos, y luego lo evaluamos en la práctica para instancias usadas en el paper que tomamos como referencia. Hemos comprobado, como se esperaba para un problema NP-Hard, que el modelo tiene una alta complejidad computacional y su resolución es lenta, incluso para discretizaciones no muy finas. No obstante, hemos visto que la grilla completa parece más eficiente computacionalmente que la grilla damero, y que la cantidad de grafos de una instancia y la de drones disponibles infuyen más en el tiempo de construcción del modelo que el tipo de grilla elegido. Por otro lado, hemos encontrado que el factor más importante en el tiempo de resolución parece ser la cantidad de grafos. También encontramos y resolvimos problemas de tiempo de cómputo y de uso de memoria, comprobando en el proceso la importancia de la forma de armar el modelo matemático y cómo se implementa luego en el código.

Quedan varias posibles continuaciones de este trabajo en el tintero. Ya mencionamos que se podría intentar un enfoque de tipo *warm start* para construir soluciones progresivamente a través de refinamientos, o bien precalcular una parte del problema y comparar los nuevos tiempos de resolución. También se podría estudiar qué pasa si el problema se generaliza a uno con varios vehículos base, donde cada uno tiene algunos drones disponibles para comenzar, aunque este problema sería computacionalmente más complejo que el ya estudiado. Otras continuaciones posibles son estudiar algún caso de aplicación realista combinando los modelos programados con algún mapa público como Open Street Map, o implementar alguna heurística como se hace en el paper de referencia y otros trabajos donde se optimizan operaciones con drones.

El modelo que construimos puede ser utilizado para algunos problemas de la vida real ya abordados en la literatura, como exploración con vehículos autónomos submarinos, *last mile delivery* u observación a distancia de diversos ambientes para detectar posibles cambios o prevenir emergencias.

Bibliografía

[Amorosi et al., 2018] Amorosi, L., Chiaraviglio, L., D'Andreagiovanni, F., and Blefari-Melazzi, N. (2018). Energy-efficient mission planning of uavs for 5g coverage in rural zones. In *2018 IEEE international conference on environmental engineering (EE)*, pages 1–9. IEEE.

[Amorosi et al., 2021] Amorosi, L., Puerto, J., and Valverde, C. (2021). Coordinating drones with mothership vehicles: The mothership and drone routing problem with graphs. *Computers & Operations Research*, 136:105445.

[Amorosi et al., 2023] Amorosi, L., Puerto, J., and Valverde, C. (2023). A multiple-drone arc routing and mothership coordination problem. *Computers & Operations Research*, 159:106322.

[Avellar et al., 2015] Avellar, G. S., Pereira, G. A., Pimenta, L. C., and Iscold, P. (2015). Multi-uav routing for area coverage and remote sensing with minimum time. *Sensors*, 15(11):27783–27803.

[Barrientos et al., 2011] Barrientos, A., Colorado, J., Cerro, J. d., Martinez, A., Rossi, C., Sanz, D., and Valente, J. (2011). Aerial remote sensing in agriculture: A practical approach to area coverage and path planning for fleets of mini aerial robots. *Journal of Field Robotics*, 28(5):667–689.

[Bell and McMullen, 2004] Bell, J. E. and McMullen, P. R. (2004). Ant colony optimization techniques for the vehicle routing problem. *Advanced engineering informatics*, 18(1):41–48.

[Cakıcı et al., 2016] Cakıcı, F., Ergezer, H., Irmak, U., and Leblebicioğlu, M. K. (2016). Coordinated guidance for multiple uavs. *Transactions of the Institute of Measurement and Control*, 38(5):593–601.

[Campbell et al., 2018] Campbell, J. F., Corberán, Á., Plana, I., and Sanchis, J. M. (2018). Drone arc routing problems. *Networks*, 72(4):543–559.

[Clarke and Wright, 1964] Clarke, G. and Wright, J. W. (1964). Scheduling of vehicles from a central depot to a number of delivery points. *Operations research*, 12(4):568–581.

[Corberán et al., 2021] Corberán, A., Eglese, R., Hasle, G., Plana, I., and Sanchis, J. M. (2021). Arc routing problems: A review of the past, present, and future. *Networks*, 77(1):88–115.

[Dantzig and Ramser, 1959] Dantzig, G. B. and Ramser, J. H. (1959). The truck dispatching problem. *Management science*, 6(1):80–91.

[Dille and Singh, 2013] Dille, M. and Singh, S. (2013). Efficient aerial coverage search in road networks. In *AIAA Guidance, Navigation, and Control (GNC) Conference*, page 5094.

[Dubins, 1957] Dubins, L. E. (1957). On curves of minimal length with a constraint on average curvature, and with prescribed initial and terminal positions and tangents. *American Journal of mathematics*, 79(3):497–516.

[Edmonds, 1973] Edmonds, J. (1973). Euler tours and the chinese postman. *Math. Programming*, 5(1):88–124.

[Ergezer and Leblebicioglu, 2013] Ergezer, H. and Leblebicioglu, K. (2013). Path planning for uavs for maximum information collection. *IEEE Transactions on Aerospace and Electronic Systems*, 49(1):502–520.

[Furini et al., 2016] Furini, F., Persiani, C. A., and Toth, P. (2016). The time dependent traveling salesman planning problem in controlled airspace. *Transportation Research Part B: Methodological*, 90:38–55.

[Garone et al., 2010] Garone, E., Naldi, R., Casavola, A., and Frazzoli, E. (2010). Cooperative mission planning for a class of carrier-vehicle systems. In *49th IEEE Conference on Decision and Control (CDC)*, pages 1354–1359. IEEE.

[Golden and Wong, 1981] Golden, B. L. and Wong, R. T. (1981). Capacitated arc routing problems. *Networks*, 11(3):305–315.

[Gouveia et al., 2010] Gouveia, L., Mourão, M. C., and Pinto, L. S. (2010). Lower bounds for the mixed capacitated arc routing problem. *Computers & Operations Research*, 37(4):692–699.

[Gu et al., 2013] Gu, Z., Hua, Q.-S., Wang, Y., and Lau, F. C. (2013). Reducing information gathering latency through mobile aerial sensor network. In *2013 Proceedings IEEE INFOCOM*, pages 656–664. IEEE.

[Hwang, 2002] Hwang, H.-S. (2002). An improved model for vehicle routing problem with time constraint based on genetic algorithm. *Computers & Industrial Engineering*, 42(2-4):361–369.

[Ji et al., 2015] Ji, X., Wang, X., Niu, Y., and Shen, L. (2015). Cooperative search by multiple unmanned aerial vehicles in a nonconvex environment. *Mathematical Problems in Engineering*, 2015(1):196730.

[Joshi and Kaur, 2015] Joshi, S. and Kaur, S. (2015). Nearest neighbor insertion algorithm for solving capacitated vehicle routing problem. In *2015 2nd International Conference on Computing for Sustainable Global Development (INDIACom)*, pages 86–88. IEEE.

[Li et al., 2016] Li, D., Wang, X., and Sun, T. (2016). Energy-optimal coverage path planning on topographic map for environment survey with unmanned aerial vehicles. *Electronics Letters*, 52(9):699–701.

[Lin and Goodrich, 2014] Lin, L. and Goodrich, M. A. (2014). Hierarchical heuristic search using a gaussian mixture model for uav coverage planning. *IEEE transactions on cybernetics*, 44(12):2532–2544.

[Liu et al.,] Liu, F., Lu, C., Gui, L., Zhang, Q., Tong, X., and Yuan, M. Heuristics for vehicle routing problem: A survey and recent advances. arxiv 2023. *arXiv preprint arXiv:2303.04147*.

[Luo et al., 2017] Luo, Z., Liu, Z., and Shi, J. (2017). A two-echelon cooperated routing problem for a ground vehicle and its carried unmanned aerial vehicle. *Sensors*, 17(5):1144.

[Mei-Ko, 1962] Mei-Ko, K. (1962). Graphic programming using odd or even points. *Chinese Math*, 1:237–277.

[Miller et al., 1960] Miller, C. E., Tucker, A. W., and Zemlin, R. A. (1960). Integer programming formulation of traveling salesman problems. *Journal of the ACM (JACM)*, 7(4):326–329.

[Orloff, 1974] Orloff, C. S. (1974). A fundamental problem in vehicle routing. *Networks*, 4(1):35–64.

[Osman, 1993] Osman, I. H. (1993). Metastrategy simulated annealing and tabu search algorithms for the vehicle routing problem. *Annals of operations research*, 41(4):421–451.

[Otto et al., 2018] Otto, A., Agatz, N., Campbell, J., Golden, B., and Pesch, E. (2018). Optimization approaches for civil applications of unmanned aerial vehicles (uavs) or aerial drones: A survey. *Networks*, 72(4):411–458.

[Sujit et al., 2009] Sujit, P., Sousa, J., and Pereira, F. L. (2009). Uav and auvs coordination for ocean exploration. In *Oceans 2009-Europe*, pages 1–7. IEEE.

[Tang and Wang, 2006] Tang, L. and Wang, X. (2006). Iterated local search algorithm based on very large-scale neighborhood for prize-collecting vehicle routing problem. *The International Journal of Advanced Manufacturing Technology*, 29(11):1246–1258.

[Viguria et al., 2010] Viguria, A., Maza, I., and Ollero, A. (2010). Distributed service-based cooperation in aerial/ground robot teams applied to fire detection and extinguishing missions. *Advanced Robotics*, 24(1-2):1–23.

[Yakıcı et al., 2024] Yakıcı, E., Karatas, M., Eriskin, L., and Cicek, E. (2024). Location and routing of armed unmanned aerial vehicles and carrier platforms against mobile targets. *Computers & Operations Research*, 169:106727.